

LTI Systems: Qualitative Analysis of the Step Response

Table of Contents

Introduction.....	1
With a Little Help from the 'help' Command!.....	1
Step Response of First Order Systems.....	9
Strictly Proper Systems.....	9
The Step Response Characteristic Parameters.....	11
Not Strictly Proper Systems.....	13
The Step Response Characteristic Parameters.....	14
Step Response of Second Order Systems.....	15
Strictly Proper Systems - Complex Poles, no Zeros.....	15
The Step Response Characteristic Parameters.....	17
Not Strictly Proper Systems - Complex Poles, one Zeros.....	18
The Step Response Characteristic Parameters.....	19
Systems of Order greater than 2 - Dominant Poles Approximation.....	20
Example - A Real Dominant Pole.....	21
The Dominant Pole Approximation.....	23
Example 2 - A Real Dominant Pole and a Zero close to the Origin.....	26
The Dominant Pole Approximation.....	27
Example 3 - Complex Dominant Poles.....	30
The Dominant Poles Approximation.....	32
Final Example - A Set of Dominant Poles and Zeros.....	35
The Dominant Poles Approximation.....	36

Introduction

In this live script, we will illustrate with examples the **qualitative behaviour** of the **step response** for **proper, asymptotically stable LTI systems** of first, second and higher order, without or with zeros, varying the position of the poles or the position of the zeros if present. Furthermore, we will illustrate how to evaluate the **characteristic parameters of the step response** (i.e. rise time, settling time, etc.) using the functions available in MATLAB's **Control System Toolbox**.

```
clear
clc
```

With a Little Help from the 'help' Command!

Here is the documentation, available with the help command, related to the Control System Toolbox function we will use in this live script

```
help tf

tf – Transfer function model
Use tf to create real-valued or complex-valued transfer function models,
or to convert dynamic system models to transfer function form.

Creation
Syntax
    sys = tf(numerator,denominator)
    sys = tf(numerator,denominator,ts)
    sys = tf(numerator,denominator,ltiSys)

    sys = tf(m)
```

```
sys = tf(____,Name,Value)
```

```
sys = tf(ltiSys)
```

```
sys = tf(ltiSys,component)
```

```
s = tf('s')
```

```
z = tf('z',ts)
```

Input Arguments

numerator – Numerator coefficients of the transfer function

row vector | Ny-by-Nu cell array of row vectors

denominator – Denominator coefficients of the transfer function

row vector | Ny-by-Nu cell array of row vectors

ts – Sample time

scalar

ltiSys – Dynamic system

dynamic system model | model array

m – Static gain

scalar | matrix

component – Component of identified model

'measured' (default) | 'noise' | 'augmented'

Output Arguments

sys – Output system model

tf model object | genss model object | uss model object

Properties

Numerator – Numerator coefficients

row vector | Ny-by-Nu cell array of row vectors

Denominator – Denominator coefficients

row vector | Ny-by-Nu cell array of row vectors

Variable – Transfer function display variable

's' (default) | 'z' | 'p' | 'q' | 'z⁻¹' | 'q⁻¹'

IODelay – Transport delay

0 (default) | scalar | Ny-by-Nu array

InputDelay – Input delay

0 (default) | scalar | Nu-by-1 vector

OutputDelay – Output delay

0 (default) | scalar | Ny-by-1 vector

Ts – Sample time

0 (default) | positive scalar | -1

TimeUnit – Time variable units

'seconds' (default) | 'nanoseconds' | 'microseconds' |

'milliseconds' | 'minutes' | 'hours' | 'days' | 'weeks' | 'months' |

'years' | ...

InputName – Input channel names

'' (default) | character vector | cell array of character vectors

InputUnit – Input channel units

'' (default) | character vector | cell array of character vectors

InputGroup – Input channel groups

structure

OutputName – Output channel names

'' (default) | character vector | cell array of character vectors

OutputUnit – Output channel units

'' (default) | character vector | cell array of character vectors

OutputGroup – Output channel groups

structure

Name – System name

'' (default) | character vector

Notes – User-specified text

{ } (default) | character vector | cell array of character vectors

UserData – User-specified data

[] (default) | any MATLAB data type

SamplingGrid – Sampling grid for model arrays

structure array

Object Functions

exp	- Create pure continuous-time delays
totaldelay	- Total combined I/O delays for LTI model
balancmr	- (Not recommended) Balanced model truncation via square root method
bilin	- Multivariable bilinear transform of frequency (s or z)
dcgainmr	- (Not recommended) Reduced order model
hankelmr	- (Not recommended) Hankel minimum degree approximation (MDA) without balancing
hankelsv	- (Not recommended) Compute Hankel singular values for stable/unstable or continuous/discrete system
ltiarray2uss	- Compute uncertain system bounding given LTI ss array
schurmr	- (Not recommended) Balanced model truncation via Schur method
sectf	- State-space sector bilinear transformation
absorbDelay	- Replace time delays by poles at $z = 0$ or phase shift
bode	- Bode frequency response of dynamic system
bodemag	- Magnitude-only Bode plot of frequency response
c2d	- Convert model from continuous to discrete time
canon	- (Not recommended) Canonical state-space realization
chgTimeUnit	- Change time units of dynamic system
connect	- Block diagram interconnections of dynamic systems
d2c	- Convert model from discrete to continuous time
d2d	- Resample discrete-time model
dcgain	- Low-frequency (DC) gain of LTI system
delay2z	- Replace delays of discrete-time TF, SS, or ZPK models by poles at $z=0$, or replace delays of FRD models by phase shift
evalfr	- Evaluate system response at specific frequency
findop	- Compute operating condition from specifications
impulse	- Impulse response plot of dynamic system; impulse response data
interp	- Interpolate FRD model
iopzmap	- Plot pole-zero map for input-output pairs of dynamic system using default options
isct	- Determine if dynamic system model is in continuous time
isdt	- Determine if dynamic system model is in discrete time
isproper	- Determine if dynamic system model is proper
isstable	- Determine if dynamic system model is stable
lsim	- Compute time response simulation data of dynamic system to arbitrary inputs
nyquist	- Nyquist response of dynamic system
order	- Query model order
pole	- Poles of dynamic system
pzmap	- Pole-zero map of dynamic system
repsys	- Replicate and tile models
set	- Set or modify model properties
ssdata	- Access state-space model data
step	- Step response of dynamic system
tfdata	- Access transfer function data
zero	- Zeros and gain of SISO dynamic system
zpkdata	- Access zero-pole-gain data
allmargin	- Gain margin, phase margin, delay margin, and crossover frequencies
bandwidth	- Frequency response bandwidth
covar	- Output and state covariance of system driven by white noise
damp	- Natural frequency and damping ratio
getGainCrossover	- Crossover frequencies for specified gain
getPassiveIndex	- Compute passivity index of linear system
getPeakGain	- Peak gain of dynamic system frequency response
getSectorCrossover	- Crossover frequencies for sector bound
getSectorIndex	- Compute conic-sector index of linear system
initial	- System response to initial states of state-space model
isPassive	- Check passivity of linear systems
margin	- Gain margin, phase margin, and crossover frequencies
nichols	- Nichols response of dynamic system
norm	- Norm of linear model
passiveplot	- Compute or plot passivity index as function of frequency
sectorplot	- Compute or plot sector index as function of frequency
sigma	- Singular values of frequency response of dynamic system
stepinfo	- Rise time, settling time, and other step-response characteristics
tzero	- Invariant zeros of linear system
looptune	- Tune fixed-structure feedback loops
looptuneSetup	- Convert tuning setup for looptune to tuning setup for systune
loopview	- Graphically analyze MIMO feedback loops
pidtune	- PID tuning algorithm for linear plant model
rlocus	- Root locus of dynamic system
systune	- Tune fixed-structure control systems modeled in MATLAB
balreal	- Balanced state-space realization

dssdata	- Extract descriptor state-space data
freqsep	- Slow-fast decomposition
mechssdata	- Access second-order sparse state-space model data
minreal	- Minimal realization or pole-zero cancellation
modalsep	- Compute modal decomposition
piddata	- Access coefficients of parallel-form PID controller
piddata2	- Access coefficients of parallel-form 2-DOF PID controller
pidstddata	- Access coefficients of standard-form PID controller
pidstddata2	- Access coefficients of standard-form 2-DOF PID controller
sparssdata	- Access first-order sparse state-space model data
spectralfact	- Spectral factorization of linear systems
stabsep	- Stable-unstable decomposition
upsample	- Upsample discrete-time models
balred	- (Not recommended) Model order reduction
hsvd	- (Not recommended) Hankel singular values of dynamic system
hsvplot	- (Not recommended) Plot Hankel singular values
dksynperf	- (Not recommended) Robust H_{∞} performance optimized by dksyn
loopmargin	- (Not recommended) Stability margin analysis of LTI and Simulink feedback loops
mktito	- Partition LTI system into two-input/two-output system
modreal	- (Not recommended) Modal form realization and projection
ncfmr	- (Not recommended) Model reduction from normalized coprime factorization
wcmargin	- (Not recommended) Worst-case disk stability margins of uncertain feedback loops
wcsens	- (Not recommended) Calculate worst-case sensitivity and complementary sensitivity functions of plant-controller feedback loop
augw	- Plant augmentation for weighted mixed-sensitivity H_{∞} and H_2 loop-shaping design
bstmr	- (Not recommended) Balanced stochastic model truncation (BST) via Schur method
diskmargin	- Disk-based stability margins of feedback loops
diskmarginplot	- Visualize disk-based stability margins
gapmetric	- Gap metric and Vinnicombe (nu-gap) metric for distance between two systems
h2hinfosyn	- Mixed H_2/H_{∞} synthesis with regional pole placement constraints
h2syn	- Compute H_2 optimal controller
hinffc	- Full-control H-infinity synthesis
hinffi	- Full-information H-infinity synthesis
hinfnorm	- H_{∞} norm of dynamic system
hinfstruct	- H_{∞} tuning of fixed-structure controllers
hinfosyn	- Compute H-infinity optimal controller
lncf	- Left normalized coprime factorization
loopsens	- Sensitivity functions of plant-controller feedback loop
loopsyn	- Loop-shaping controller design with tradeoff between performance and robustness
ltrsyn	- LQG loop transfer-function recovery (LTR) control synthesis
mixsyn	- Mixed-sensitivity H_{∞} synthesis method for robust control loop-shaping design
ncfmarg	- Calculate normalized coprime stability margin of plant-controller feedback loop
ncfsyn	- Loop shaping design using Glover-McFarlane method
rncf	- Right normalized coprime factorization
robgain	- Robust performance of uncertain system
robstab	- Robust stability of uncertain system
robustperf	- (Not recommended) Robust performance margin of uncertain multivariable system
robuststab	- (Not recommended) Calculate robust stability margins of uncertain multivariable system
sdhinfnorm	- Compute L_2 norm of continuous-time system in feedback with discrete-time system
sdhinfosyn	- Compute H_{∞} controller for sampled-data system
sdlsim	- Time response of sampled-data feedback system
slowfast	- (Not recommended) Slow and fast modes decomposition
ucover	- Fit uncertain model to set of LTI responses
wcdiskmargin	- Worst-case disk-based stability margins of uncertain feedback loops
wcdiskmarginplot	- Visualize worst-case disk-based stability margins
wcgain	- Worst-case gain of uncertain system
wcsigmaplot	- Plot worst-case gain of uncertain system
conj	- Form model with complex conjugate coefficients
ctranspose	- Conjugate dynamic system model
squeeze	- Remove singleton dimensions for umat objects
permute	- Rearrange array dimensions in model arrays
isfinite	- Determine if model has finite coefficients
isreal	- Determine if model has real-valued coefficients
hasInternalDelay	- Determine if model has internal delays
hasdelay	- True for linear model with time delays
isstatic	- Determine if model is static or dynamic
isempty	- Determine whether dynamic system model is empty
issiso	- Determine if dynamic system model is single-input/single-output (SISO)
ndims	- Query number of dimensions of dynamic system model or model array

nblocks	- Number of control design blocks in generalized LTI model or generalized matrix
nmodels	- Number of models in model array
isParametric	- Determine if model has tunable parameters
append	- Group models by appending their inputs and outputs
blkdiag	- Block-diagonal concatenation of models
get	- Access model property values
getValue	- Current value of generalized model
repmat	- Replicate and tile array
reshape	- Change shape of model array
size	- Query output/input/array dimensions of input-output model and number of frequencies of FRD model
stack	- Build model array by stacking models or model arrays along array dimensions
voidModel	- Mark missing or irrelevant models in model array
feedback	- Feedback connection of multiple models
getBlockValue	- Get current value of Control Design Block in Generalized Model
imp2exp	- Convert implicit linear relationship to explicit input-output relation
inv	- Invert dynamic system models
lft	- Generalized feedback interconnection of two models (Redheffer star product)
parallel	- Parallel connection of two models
replaceBlock	- Replace or update control design blocks in generalized model
rsampleBlock	- Randomly sample Control Design blocks in generalized model
sampleBlock	- Sample Control Design blocks in generalized model
series	- Series connection of two models
setBlockValue	- Modify value of Control Design Block in Generalized Model
showBlockValue	- Display current values of Control Design Blocks in Generalized Model
showTunable	- Display current value of tunable Control Design Blocks in Generalized Model
sminreal	- Eliminates structurally disconnected states, delays, and blocks
getNominal	- Nominal value of uncertain model
gridureal	- Grid ureal parameters uniformly over their range
lftdata	- Decompose uncertain objects into fixed certain and normalized uncertain parts
simplify	- Simplify representation of uncertain object
uscale	- Scale uncertainty of block or system
usubs	- Substitute given values for uncertain elements of uncertain objects

Examples

SISO Transfer Function Model
 Discrete-Time SISO Transfer Function Model
 Second-Order Transfer Function from Damping Ratio and Natural Frequency
 Discrete-Time MIMO Transfer Function Model
 Concatenate SISO Transfer Functions into MIMO Transfer Function Model
 Transfer Function Model Using Rational Expression
 Discrete-Time Transfer Function Model Using Rational Expression
 Transfer Function Model with Inherited Properties
 Array of Transfer Function Models
 Convert State-Space Model to Transfer Function
 Extract Transfer Functions from Identified Model
 Specify Input and Output Names for MIMO Transfer Function Model
 Specify Polynomial Ordering in Discrete-Time Transfer Function
 Tunable Low-Pass Filter
 Static Gain MIMO Transfer Function Model

See also `filt`, `frd`, `get`, `set`, `ss`, `tfdata`, `zpk`, `genss`, `realp`, `genmat`, `tunableTF`

Introduced in Control System Toolbox before R2006a
 Documentation for `tf`
 Other uses of `tf`

help `step`

step – Step response of dynamic system
`step` computes the step response to a step change in input value from `U` to `U + dU` after `td` time units.

Syntax

```

[y,tOut] = step(sys)
[y,tOut] = step(sys,t)
[y,tOut] = step(sys,t,p)
  
```

```
[y,tOut,x] = step(____)
[y,tOut,x,ysd] = step(____)
[y,tOut,x,~,pOut] = step(sys,t,p)
```

```
[y,tOut] = step(____,config)
```

```
step(____)
```

Input Arguments

```
sys – Dynamic system
    dynamic system model | model array
t – Time steps
    positive scalar | two-element vector | vector | []
p – LPV model parameter trajectory
    matrix | function handle
config – Response configuration
    RespConfig object
```

Output Arguments

```
y – Step response data
    array
tOut – Times at which step response is computed
    vector
x – State trajectories
    array
ysd – Standard deviation of step response
    array
pOut – Parameter trajectories
    array
```

Examples

```
Step Response of Dynamic System
Step Response of Discrete-Time System
Step Response at Specified Times
Step Response Plot of MIMO Systems
Compare Responses of Multiple Systems
Step Response of Systems in a Model Array
Step Response Data
Step Response Plot of Feedback Loop with Delay
Response to Custom Step Input
Step Responses of Identified Models with Confidence Regions
Step Response of Identified Time-Series Model
Validate Linearization of Identified Nonlinear ARX Model
Step Response of LPV Model
```

See also `impulse`, `RespConfig`, `initial`, `lsim`, `stepplot`,
Linear System Analyzer

Introduced in Control System Toolbox before R2006a
Documentation for `step`
Other uses of `step`

help `RespConfig`

RespConfig – Options for step or impulse responses

Use a `RespConfig` object to specify options for plotting step responses (`step`, `stepplot`), impulse responses (`impulse`, `impulseplot`), and initial responses (`initial` and `initialplot`).

Creation

```
Syntax
respOpt = RespConfig
respOpt = RespConfig(Name=Value)
```

Properties

```
Bias – Baseline input signal value
    0 (default) | scalar | vector | 'u0'
```

Amplitude – Input level change
1 (default) | scalar | vector
Delay – Step or impulse delay
0 (default) | nonnegative scalar
InitialState – Initial state values
[] (default) | vector | 'x0' | initialCondition object
InitialParameter – Initial parameter values for LPV models
[] (default) | scalar | vector

Examples

Configure Options for Step Response
Configure Options for Impulse Response
Simulate State-Space Model Relative to Model Offsets

See also `impulse`, `impzplot`, `step`, `stepplot`

Introduced in Control System Toolbox in R2023a
Documentation for `RespConfig`
Other uses of `RespConfig`

help `stepplot`

--- help for `controllib.chart.StepPlot` ---

`controllib.chart.StepPlot` – Plot step response of dynamic system

The `stepplot` function plots the step response of a dynamic system model and returns a `StepPlot` chart object.

Creation

Syntax

```
sp = stepplot(sys)
sp = stepplot(sys1,sys2,...,sysN)
sp = stepplot(sys1,LineStyle1,...,sysN,LineStyleN)
sp = stepplot(____,t)
sp = stepplot(____,t,p)

sp = stepplot(____,config)

sp = stepplot(____,plotoptions)

sp = stepplot(parent,____)
```

Input Arguments

`sys` – Dynamic system
dynamic system model | model array
`LineStyle` – Line style, marker, and color
string | character vector
`t` – Time steps
positive scalar | two-element vector | vector | []
`p` – LPV model parameter trajectory
matrix | function handle
`config` – Response configuration
RespConfig object
`plotoptions` – Time response plot options
timeoptions object
`parent` – Parent container
Figure object (default) | TiledChartLayout object |
UIFigure object | UIGridLayout object | UIPanel object |
UITab object

Properties

`Responses` – Model responses
StepResponse object | array of StepResponse objects
`Characteristics` – Response characteristics
CharacteristicsManager object
`TimeUnit` – Time units
"seconds" | "milliseconds" | "minutes" | ...

Normalize – Option to normalize plot
"off" (default) | on/off logical value
Visible – Chart visibility
"on" (default) | on/off logical value
IOGrouping – Grouping of inputs and outputs pairs
"none" (default) | "inputs" | "outputs" | "all"
InputVisible – Option to display inputs
on/off logical value | array of on/off logical values
OutputVisible – Option to display outputs
on/off logical value | array of on/off logical values

Object Functions

setoptions – (Not recommended) Set options for linear analysis plot object
getoptions – (Not recommended) Get options for linear analysis plot object
addResponse – Add dynamic system response to existing response plot

Examples

Customize Step Plot
Display Normalized Response on Step Plot
Plot Step Responses of Identified Models with Confidence Region
Customized Step Response Plot at Specified Time
Plot Step Response of Nonlinear Identified Model

See also `step`, `timeoptions`

Introduced in Control System Toolbox before R2006a
Documentation for `controllib.chart.StepPlot`

help `stepinfo`

stepinfo – Rise time, settling time, and other step-response characteristics
stepinfo lets you compute step-response characteristics for a dynamic system model or for an array of step-response data.

Syntax

S = **stepinfo**(sys)

S = **stepinfo**(y,t)
S = **stepinfo**(y,t,yfinal)
S = **stepinfo**(y,t,yfinal,yinit)

S = **stepinfo**(____,'SettlingTimeThreshold',ST)
S = **stepinfo**(____,'RiseTimeLimits',RT)

Input Arguments

sys – Dynamic system
dynamic system model
y – Step-response data
vector | array
t – Time vector
vector
yfinal – Steady-state value
scalar | array
yinit – Initial value
scalar | array
ST – Settling time threshold
0.02 (default) | scalar between 0 and 1
RT – Rise time thresholds
[0.1 0.9] (default) | 2-element row vector

Output Arguments

S – Step-response characteristics
structure

Examples

Step-Response Characteristics of Dynamic System
Step-Response Characteristics of MIMO System

Specify Percentage for Settling Time or Rise Time
Step-Response Characteristics from Response Data
Difference Between Transient Time and Settling Time for Step Responses
Step Response Characteristics for Data with Initial Offset

See also `step`, `lsiminfo`

Introduced in Control System Toolbox in R2006a
Documentation for `stepinfo`
Other uses of `stepinfo`

Step Response of First Order Systems

Strictly Proper Systems

Let's consider

$$G(s) = \frac{\mu}{1 + s\tau}, \quad \mu > 0, \tau > 0$$

Remeber:

$$t_r \approx 2.2\tau, \quad t_d \approx 0.7\tau, \quad t_{s0.01} \approx 4.6\tau \quad (+)$$

Let's explore what happens when the pole $p = -\frac{1}{\tau}$ moves, starting close to the origin in $s = 0$ and moving away from the origin towards $-\infty$ on the real axis

Let's choose some pole positions in the Left Half Plane of the complex frequency s (LHP for short).

```
Np = 7; % 7 different pole positions -- we have 7 available colors for plots
tau_p = logspace(-1,1, Np); % Np time constants tau, logarithmically spaced,
                                % starting from -1e-1 and ending with -1e+1

muG = 2+8*rand(1); % a positive value in the range [2, 10]
```

Pay attention on this function:

```
help stepinfo
```

stepinfo – Rise time, settling time, and other step-response characteristics
stepinfo lets you compute step-response characteristics for a dynamic system model or for an array of step-response data.

Syntax

```

S = stepinfo(sys)

S = stepinfo(y,t)
S = stepinfo(y,t,yfinal)
S = stepinfo(y,t,yfinal,yinit)

S = stepinfo(____,'SettlingTimeThreshold',ST)
S = stepinfo(____,'RiseTimeLimits',RT)

```

Input Arguments

```

sys - Dynamic system
      dynamic system model
y - Step-response data
    vector | array
t - Time vector
    vector
yfinal - Steady-state value
        scalar | array
yinit - Initial value
        scalar | array
ST - Settling time threshold
    0.02 (default) | scalar between 0 and 1
RT - Rise time thresholds
    [0.1 0.9] (default) | 2-element row vector

```

Output Arguments

```

S - Step-response characteristics
    structure

```

Examples

```

Step-Response Characteristics of Dynamic System
Step-Response Characteristics of MIMO System
Specify Percentage for Settling Time or Rise Time
Step-Response Characteristics from Response Data
Difference Between Transient Time and Settling Time for Step Responses
Step Response Characteristics for Data with Initial Offset

```

See also `step`, `lsiminfo`

Introduced in Control System Toolbox in R2006a
Documentation for `stepinfo`
Other uses of `stepinfo`

We will use it to have an accurate estimation of the characteristic parameters of the step response and we'll compare th results with the approximate formulas in [Eq. \(+\)](#).

```

G1sN = struct('tf',cell(1), ...
              'y_step', cell(1), ...
              't_step', cell(1)); % we'll store each different transfer function
G1sStepINFO = cell(Np); % and the step response parameters

figure('Units','normalized', 'Position',[0.1, 0.1,0.99,0.99]);
% let's create a blank figure, as wide as possible
currAx = gca; % retrieve the current axes handler
hold on;

Tfinal = 1.5*5*max(tau_p); % highest than the longest settling time
for n=Np:-1:1
    % for each pole position
    tau = tau_p(n);
    currGs = tf(muG, [tau +1] ); % 1st order, NO zero
    [Ystep, Tstep] = step(currGs, Tfinal); % plot the current step response
    plot(currAx,Tstep, Ystep, 'LineWidth', 1.5);
    G1sN(n).tf = currGs;

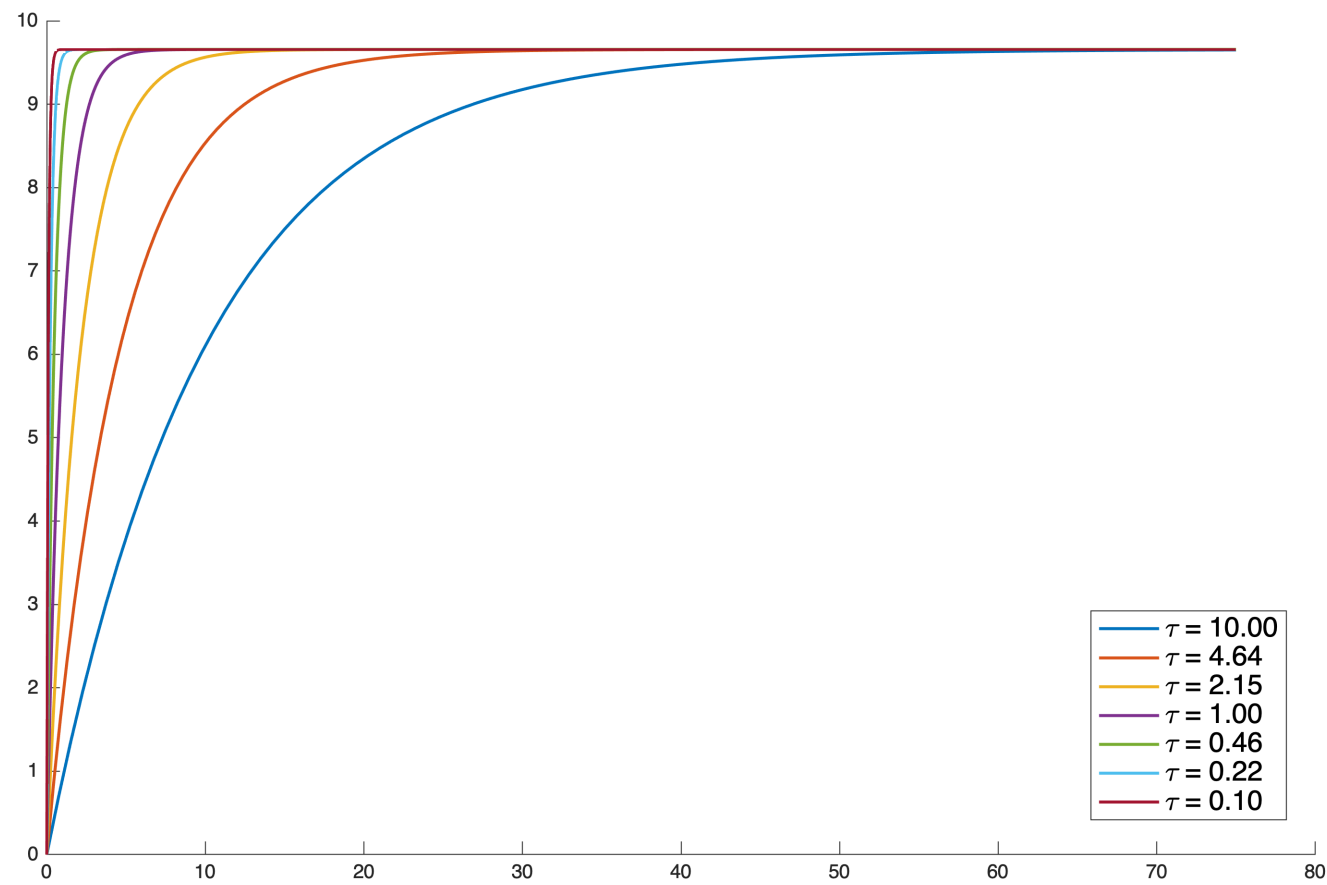
```

```

G1sN(n).y_step = Ystep;
G1sN(n).t_step = Tstep;
G1sStepINFO{n} = stepinfo(currGs, "SettlingTimeThreshold", 0.01);
end % for n

formatSpec = '%.2f'; % see 'doc num2str'
legend(['\tau = ', num2str(tau_p(7), formatSpec) ], ...
       ['\tau = ', num2str(tau_p(6), formatSpec) ], ...
       ['\tau = ', num2str(tau_p(5), formatSpec) ], ...
       ['\tau = ', num2str(tau_p(4), formatSpec) ], ...
       ['\tau = ', num2str(tau_p(3), formatSpec) ], ...
       ['\tau = ', num2str(tau_p(2), formatSpec) ], ...
       ['\tau = ', num2str(tau_p(1), formatSpec) ], ...
       'Location', 'best', 'FontSize', 14)

```



Notice: the system with $\tau = 10$ has the slowest step response, whereas the system with $\tau = 0.1$ is the fastest one.

The Step Response Characteristic Parameters

Let's analyse the step response parameters, comparing the expressions in [Eq. \(+\)](#) with the estimations obtained by applying the function `stepinfo( )`.

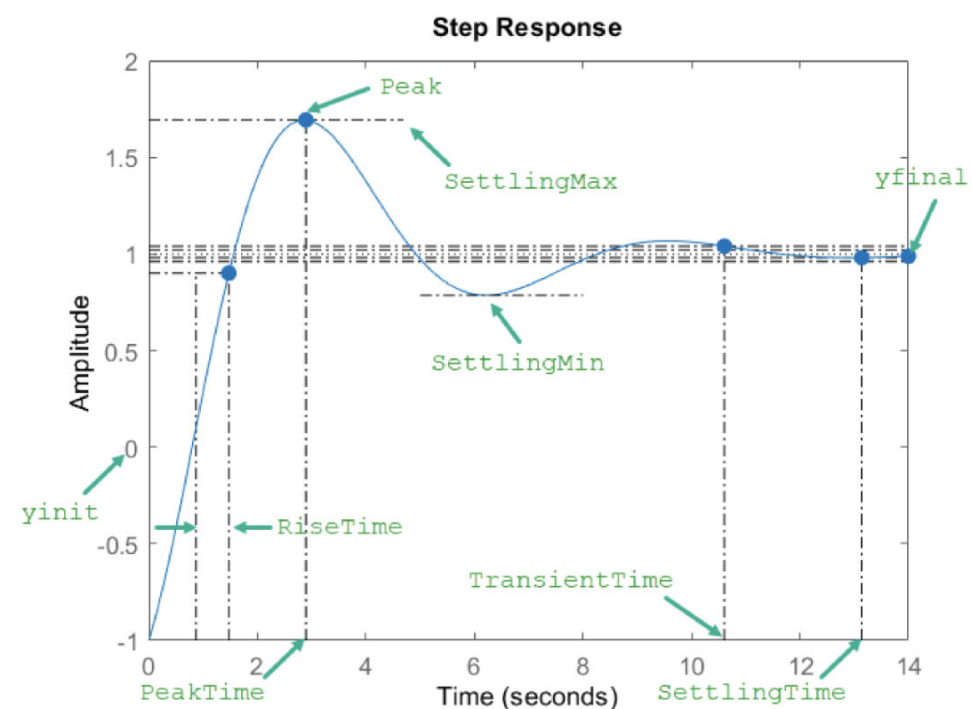
What does the function `stepinfo( )` provide?

```
disp(G1sStepINFO{1})
```

```

RiseTime: 0.2197
TransientTime: 0.4605
SettlingTime: 0.4605
SettlingMin: 8.7351
SettlingMax: 9.6510
Overshoot: 0
Undershoot: 0
Peak: 9.6510
PeakTime: 0.7322

```



Excerpt from the [MATLAB documentation](#).

Let's compare the settling time values, estimates by using [Eq. \(+\)](#) and the `stepinfo()`:

```

settlingTformula = zeros(Np,1); % initialise the array used to store
                                % the settling time values obtained
                                % using the formula
settlingT_stepinfo = zeros(size(settlingTformula));

for n = 1:Np
    settlingTformula(n) = 4.6 * tau_p(n);
    settlingT_stepinfo(n) = G1sStepINFO{n}.SettlingTime;
end % for n

settlingT_table = table(settlingTformula, settlingT_stepinfo, tau_p', ...
    'VariableNames',...
    {'formula 4.6 tau', 'estimated by stepinfo', 'tau values'});

disp(settlingT_table)

```

formula 4.6 tau	estimated by stepinfo	tau values
-----------------	-----------------------	------------

0.46	0.46052	0.1
0.99104	0.99215	0.21544
2.1351	2.1375	0.46416
4.6	4.6052	1
9.9104	9.9215	2.1544
21.351	21.375	4.6416
46	46.052	10

As you can note, the values are very close similar for the settling time of each step response.

Pay attention: this is the only scenario when both approaches give the same results. Any guess why?

Not Strictly Proper Systems

Let's consider

$$G(s) = \mu \frac{1+sT}{1+s\tau}, \quad \mu > 0, \tau > 0, T > 0$$

Let's choose a fixed pole position in the Left Half Plane of the complex frequency s (LHP for short), a static gain and explore what happens if the zero moves from LHP to RHP (Right Half Plane).

Do the formulas in [Eq. \(+\)](#) and `stepinfo()` provide similar results this time?

```
Np = 7; % 7 different pole positions -- we have 7 available colors for plots
T_z = [-fliplr(logspace(-1,1, (floor(Np/2))))], ...
    0, ...
    logspace(-1,1, (floor(Np/2)))]; % Np time constants T, logarithmically spaced,
    % starting from -1e-1 and ending with -1e+1

tau_p = 1.5;
muG = 2+8*rand(1); % a positive value in the range [2, 10]

G1sZN = struct('tf',cell(1), ...
    'y_step', cell(1), ...
    't_step', cell(1)); % we'll store each different transfer function
G1sZStepINFO = cell(Np); % and the step response parameters

figure('Units','normalized', 'Position',[0.1, 0.1,0.99,0.99]);
% let's create a blank figure, as wide as possible
currAx = gca; % retrieve the current axes handler
hold on;

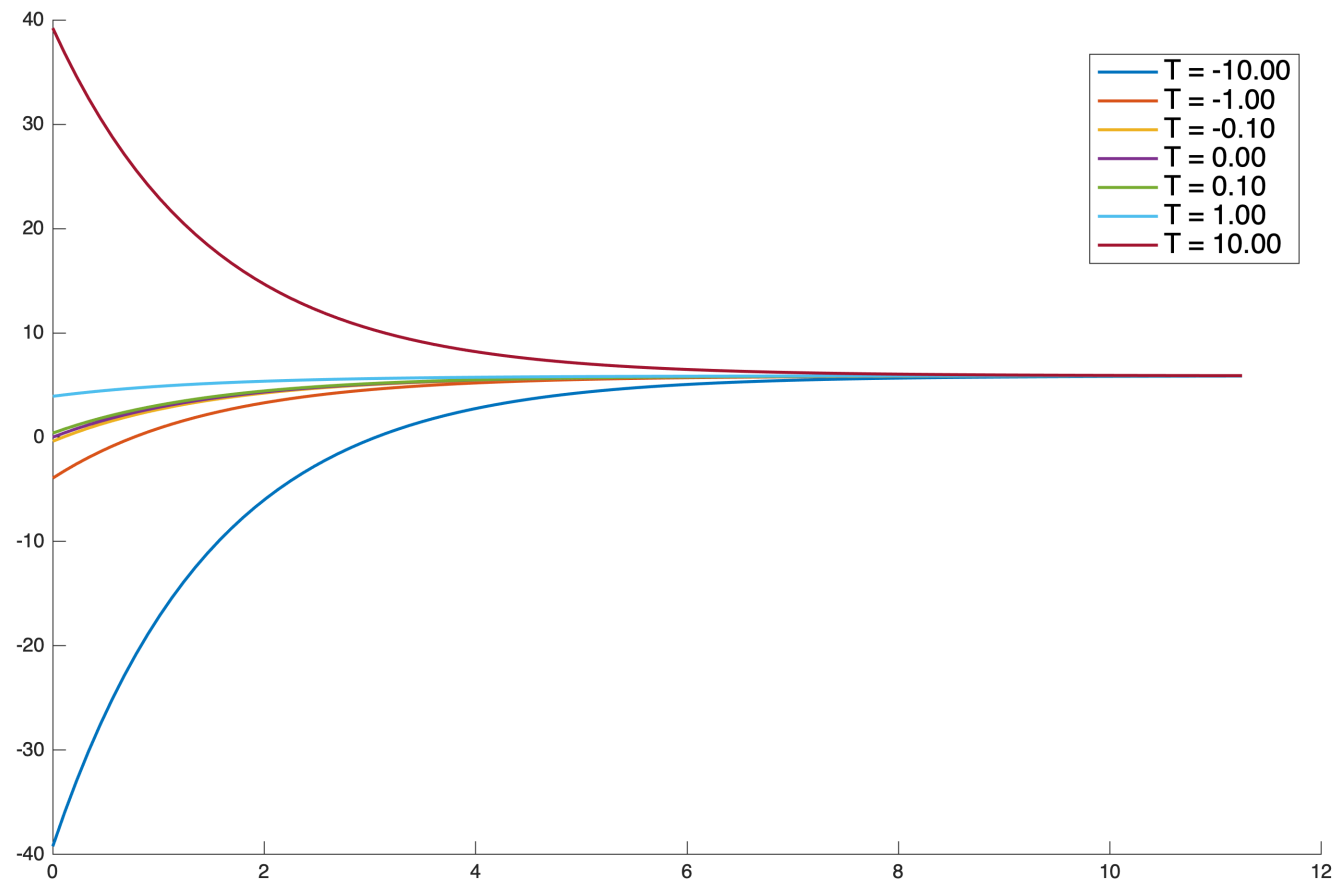
Tfinal = 1.5*5*max(tau_p); % highest than the longest settling time
for n=1:Np
    % for each pole position
    Tz = T_z(n);
    currGs = muG* tf([Tz 1], [tau_p +1] ); % 1st order, NO zero
    [Ystep, Tstep] = step(currGs, Tfinal); % plot the current step response
    plot(currAx,Tstep, Ystep, 'LineWidth', 1.5);
    G1sZN(n).tf = currGs;
    G1sZN(n).y_step = Ystep;
```

```

G1sZN(n).t_step = Tstep;
G1sZStepINFO{n} = stepinfo(currGs, "SettlingTimeThreshold", 0.01);
end % for n

formatSpec = '%.2f'; % see 'doc num2str'
legend(['T = ', num2str(T_z(1), formatSpec) ], ...
      ['T = ', num2str(T_z(2), formatSpec) ], ...
      ['T = ', num2str(T_z(3), formatSpec) ], ...
      ['T = ', num2str(T_z(4), formatSpec) ], ...
      ['T = ', num2str(T_z(5), formatSpec) ], ...
      ['T = ', num2str(T_z(6), formatSpec) ], ...
      ['T = ', num2str(T_z(7), formatSpec) ], ...
      'Location', 'best', 'FontSize', 14)

```



The Step Response Characteristic Parameters

Let's analyse the step response parameters, comparing the expressions in [Eq. \(+\)](#) with the estimations obtained by applying the function `stepinfo( )`.

```

settlingTformula = zeros(Np,1); % initialise the array used to store
                                % the settling time values obtained

```

```
% using the formula
settlingT_stepinfo = zeros(size(settlingTformula));
polePOS = -(1/tau_p)*ones(Np,1);
zeroPOS = zeros(Np,1);

for n = 1:Np
    settlingTformula(n) = 4.6 * tau_p;
    settlingT_stepinfo(n) = G1sZStepINF0{n}.SettlingTime;
    zeroPOS(n) = 1/T_z(n);
end % for n

settlingT_table = table(settlingTformula, settlingT_stepinfo, ...
                        polePOS, zeroPOS, ...
                        'VariableNames',...
                        {'formula 4.6 tau', 'estimated by stepinfo', ...
                        'pole p', 'zero z'});

disp(settlingT_table)
```

formula 4.6 tau	estimated by stepinfo	pole p	zero z
6.9	9.9634	-0.66667	-0.1
6.9	7.6741	-0.66667	-1
6.9	7.0049	-0.66667	-10
6.9	6.9078	-0.66667	Inf
6.9	6.8047	-0.66667	10
6.9	5.26	-0.66667	1
6.9	9.51	-0.66667	0.1

Step Response of Second Order Systems

Strictly Proper Systems - Complex Poles, no Zeros

Let's consider

$$G(s) = \frac{\mu}{1 + \frac{2\xi}{\omega_n}s + \frac{1}{\omega_n^2}s^2}, \quad \mu > 0, \omega_n > 0, 0 \leq \xi \leq 1$$

$$t_s \approx \frac{4.6}{\xi \omega_n} \quad (\star)$$

Let's choose a constant value for ω_n and μ , and explore what happens if the damping factor ξ moves from 0 to 1. Let's analyse the step response parameters, comparing the expressions in [Eq. \(\$\star\$ \)](#) with the estimations obtained by applying the function `stepinfo( )`.

```
Np = 7; % 7 different pole positions -- we have 7 available colors for plots
xi_val = linspace(1e-1,1-1e-1,7); % Np damping factor values

omega_n_p = 4.5;
```

```

muG = 2+8*rand(1); % a positive value in the range [2, 10]

G2sN = struct('tf',cell(1), ...
             'y_step', cell(1), ...
             't_step', cell(1)); % we'll store each different transfer function
G2sStepINFO = cell(Np,1); % and the step response parameters

```

```

Tfinal = 1.5*5*(1/omega_n_p/1e-1); % highest than the longest settling time

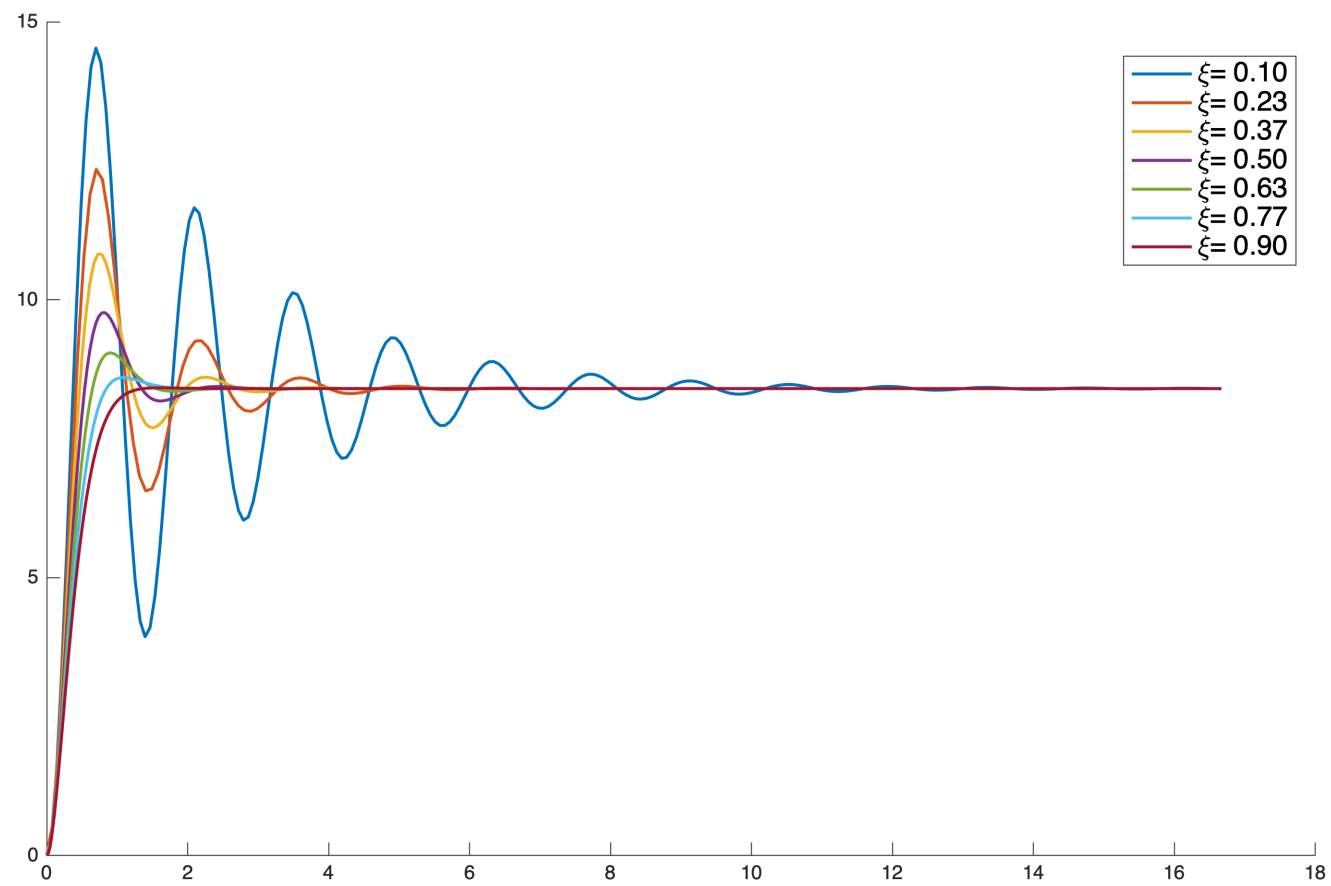
```

```

figure('Units','normalized', 'Position',[0.1, 0.1,0.99,0.99]);
% let's create a blank figure, as wide as possible
currAx = gca; % retrieve the current axes handler
hold on;

for n=1:Np
    % for each pole position
    currXi = xi_val(n);
    currGs = tf(muG, [(1/omega_n_p.^2) (2*currXi/omega_n_p) +1] ); % 2nd order, NO zero
    [Ystep, Tstep] = step(currGs, Tfinal); % plot the current step response
    plot(currAx,Tstep, Ystep, 'LineWidth', 1.5);
    G2sN(n).tf = currGs;
    G2sN(n).y_step = Ystep;
    G2sN(n).t_step = Tstep;
    G2sStepINFO{n} = stepinfo(currGs, "SettlingTimeThreshold", 0.01);
end % for n
legend(['\xi= ',num2str(xi_val(1), formatSpec) ], ...
       ['\xi= ', num2str(xi_val(2), formatSpec) ], ...
       ['\xi= ',num2str(xi_val(3), formatSpec) ], ...
       ['\xi= ',num2str(xi_val(4), formatSpec) ], ...
       ['\xi= ',num2str(xi_val(5), formatSpec) ], ...
       ['\xi= ',num2str(xi_val(6), formatSpec) ], ...
       ['\xi= ',num2str(xi_val(7), formatSpec) ],...
       'Location', 'best', 'FontSize', 14)

```



The Step Response Characteristic Parameters

Let's analyse the step response parameters, comparing the expressions in [Eq. \(+\)](#) with the estimations obtained by applying the function `stepinfo( )`.

```
settlingTformula = zeros(Np,1); % initialise the array used to store
                                % the settling time values obtained
                                % using the formula
settlingT_stepinfo = zeros(size(settlingTformula));

for n = 1:Np
    settlingTformula(n) = 4.6 / (omega_n_p * xi_val(n));
    settlingT_stepinfo(n) = G2sStepINFO{n}.SettlingTime;
end % for n

settlingT_table = table(settlingTformula, settlingT_stepinfo, xi_val', ...
                        'VariableNames', ...
                        {'formula 4.6/(omega_n * xi)', 'estimated by stepinfo', 'damping factor'});

disp(settlingT_table)
```

formula 4.6/(omega_n * xi)	estimated by stepinfo	damping factor
10.222	9.9529	0.1
4.381	4.3977	0.23333
2.7879	2.5537	0.36667
2.0444	1.9513	0.5
1.614	1.4189	0.63333
1.3333	1.4577	0.76667
1.1358	1.1395	0.9

Not Strictly Proper Systems - Complex Poles, one Zeros

Let's consider

$$G(s) = \frac{\mu(1 + sT)}{1 + \frac{2\xi}{\omega_n}s + \frac{1}{\omega_n^2}s^2}, \quad \mu > 0, \omega_n > 0, T > 0, 0 \leq \xi \leq 1$$

$$t_s \approx \frac{4.6}{\xi \omega_n} \quad (\star)$$

Let's choose a constant value for ω_n , T and μ , and explore what happens if the damping factor ξ moves from 0 to 1. Let's analyse the step response parameters, comparing the expressions in [Eq. \(★\)](#) with the estimations obtained by applying the function `stepinfo()`.

```
Np = 7; % 7 different pole positions -- we have 7 available colors for plots
xi_val = linspace(1e-1,1-1e-1,7); % Np damping factor values

omega_n_p = 4.5;
muG = 2+8*rand(1); % a positive value in the range [2, 10]
T = 1.25;

G2sN = struct('tf',cell(1), ...
              'y_step', cell(1), ...
              't_step', cell(1)); % we'll store each different transfer function
G2sStepINFO = cell(Np,1); % and the step response parameters
```

```
Tfinal = 1.5*5*(1/omega_n_p/1e-1); % highest than the longest settling time
```

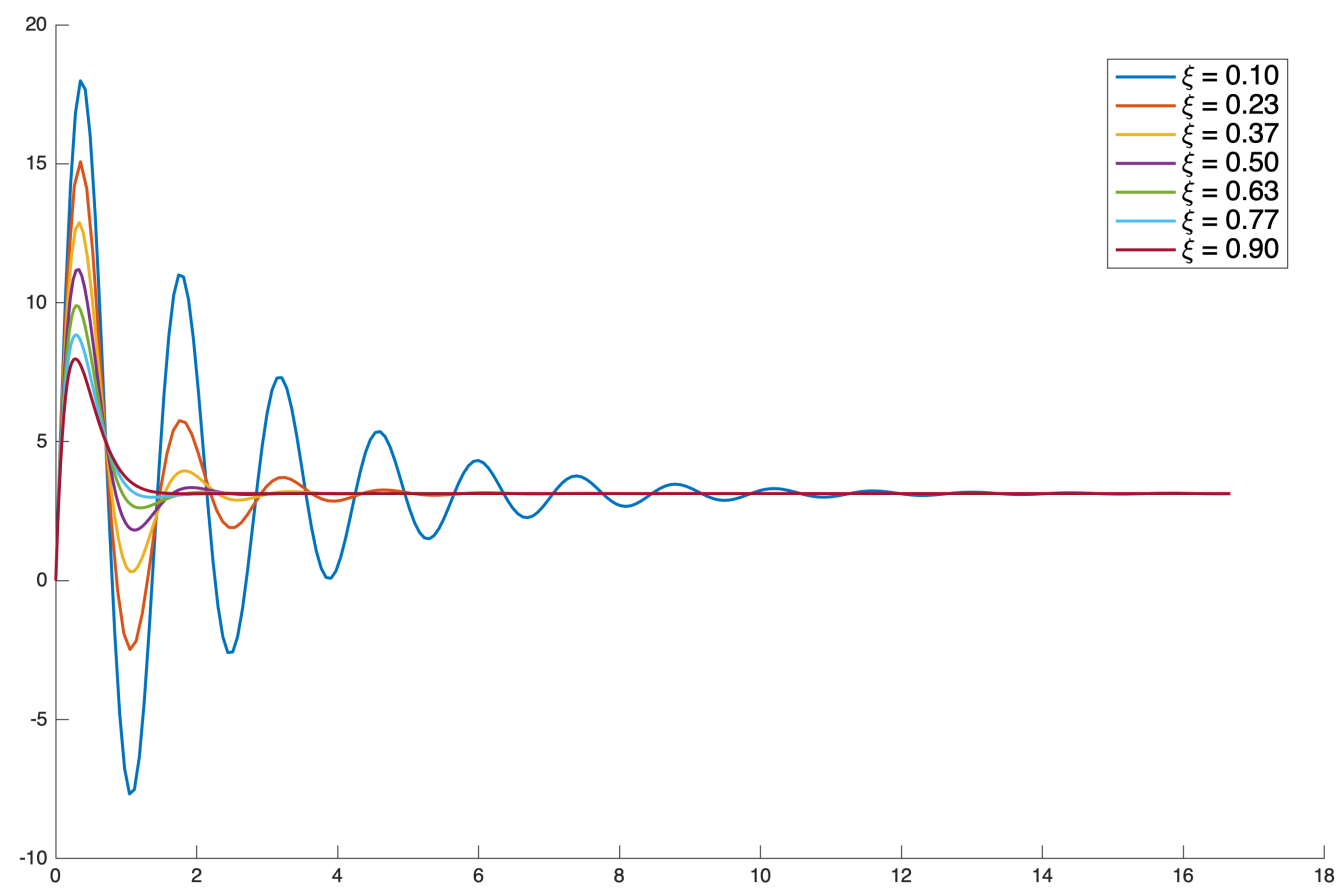
```
figure('Units','normalized', 'Position',[0.1, 0.1,0.99,0.99]);
% let's create a blank figure, as wide as possible
currAx = gca; % retrieve the current axes handler
hold on;

for n=1:Np
    % for each pole position
    currXi = xi_val(n);
```

```

currGs = muG*tf([T 1], [(1/omega_n_p.^2) (2*currXi/omega_n_p) +1] ); % 2nd order, NO zero
[Ystep, Tstep] = step(currGs, Tfinal); % plot the current step response
plot(currAx,Tstep, Ystep, 'LineWidth', 1.5);
G2sN(n).tf = currGs;
G2sN(n).y_step = Ystep;
G2sN(n).t_step = Tstep;
G2sStepINFO{n} = stepinfo(currGs, "SettlingTimeThreshold", 0.01);
end % for n
formatSpec = '%.2f'; % see 'doc num2str'
legend(['\xi = ',num2str(xi_val(1), formatSpec) ], ...
      ['\xi = ',num2str(xi_val(2), formatSpec) ], ...
      ['\xi = ',num2str(xi_val(3), formatSpec) ], ...
      ['\xi = ',num2str(xi_val(4), formatSpec) ], ...
      ['\xi = ',num2str(xi_val(5), formatSpec) ], ...
      ['\xi = ',num2str(xi_val(6), formatSpec) ], ...
      ['\xi = ',num2str(xi_val(7), formatSpec) ],...
      'Location', 'best', 'Fontsize', 14)

```



The Step Response Characteristic Parameters

Let's analyse the step response parameters, comparing the expressions in [Eq. \(+\)](#) with the estimations obtained by applying the function `stepinfo( )`.

```

settlingTformula = zeros(Np,1); % initialise the array used to store

```

```

                                % the settling time values obtained
                                % using the formula
settlingT_stepinfo = zeros(size(settlingTformula));

for n = 1:Np
    settlingTformula(n) = 4.6 /(omega_n_p*xi_val(n));
    settlingT_stepinfo(n) = G2sStepINFO{n}.SettlingTime;
end % for n

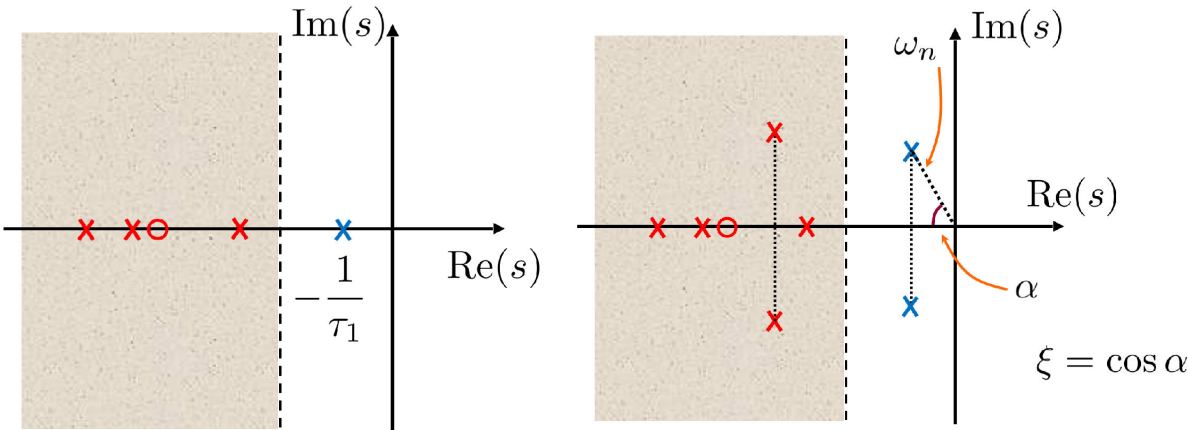
settlingT_table = table(settlingTformula, settlingT_stepinfo, xi_val', ...
                        'VariableNames',...
                        {'formula 4.6/(omega_n * xi)', 'estimated by stepinfo', 'damping factor'});

disp(settlingT_table)

```

formula 4.6/(omega_n * xi)	estimated by stepinfo	damping factor
10.222	13.826	0.1
4.381	5.6224	0.23333
2.7879	3.6155	0.36667
2.0444	2.8412	0.5
1.614	2.2725	0.63333
1.3333	1.874	0.76667
1.1358	1.4667	0.9

Systems of Order greater than 2 - Dominant Poles Approximation



When using the dominant poles approximation:

- It is essential to “preserve” the gain
- Zeros located close to the imaginary axis have to be adequately taken into account

This approximation is valid in qualitative analysis and for the initial and rough controller’s design steps. Refer to Part 7 of the course material.

Example - A Real Dominant Pole

Consider the LTI system described by the transfer function

$$G_1(s) = \frac{250 \left(1 + \frac{s}{10}\right)}{\left(1 + 20s\right) \left(1 + \frac{s}{5}\right) \left(1 + \frac{9}{20}s + \frac{s^2}{16}\right)}$$

```
clear
s = tf('s'); % the transfer function building helper

G1s = 250*(1+s/10)/((1+20*s)*(1+s/5)*(1+(9/20)*s+s^2/16)); % the tranfer function

set(G1s,"Name", "G1");
G1_staticGAIN = dcgain(G1s) % the static gain of the LTI system
```

G1_staticGAIN =
250

```
p_G1s = sort(pole(G1s), 'ascend') % the poles of the transfer function
```

p_G1s = 4×1 complex
-0.0500 + 0.0000i
-3.6000 - 1.7436i
-3.6000 + 1.7436i
-5.0000 + 0.0000i

```
zero(G1s) % the zeros of the transfer function
```

ans =
-10

The poles and zeros configuration of the system is as follows

```
hf = figure('Units','normalized', 'Position', [0.1, 0.1, 0.9, 0.9]);
hPZP = pzplot(hf, G1s);
grid on
get(hPZP)
```

AxesStyle: [1x1 controllib.chart.internal.options.AxesStyle]
Characteristics: []
FrequencyUnit: "rad/s"
HandleVisibility: 'on'
InnerPosition: [0.0543 0.0868 0.8915 0.8382]
Layout: [0x0 matlab.ui.layout.LayoutOptions]
LegendAxes: [1 1]
LegendAxesMode: "auto"
LegendLocation: "northeast"
LegendOrientation: "vertical"

```

LegendVisible: off
NextPlot: "replace"
OuterPosition: [0 0 1 1]
Parent: [1x1 Figure]
Position: [0.0543 0.0868 0.8915 0.8382]
PositionConstraint: 'outerposition'
Responses: [1x1 controllib.chart.response.PZResponse]
Subtitle: [1x1 controllib.chart.internal.options.AxesLabel]
TimeUnit: "seconds"
Title: [1x1 controllib.chart.internal.options.AxesLabel]
Units: 'normalized'
Visible: on
XLabel: [1x1 controllib.chart.internal.options.AxesLabel]
XLimits: [-10 0]
XLimitsMode: "auto"
YLabel: [1x1 controllib.chart.internal.options.AxesLabel]
YLimits: {[-2 2]}
YLimitsMode: {"auto"}

```

```
get(hPZP.Responses(1))
```

```

SourceData: [1x1 struct]
Name: "G1"
Visible: on
LegendDisplay: on
MarkerSize: 6
LineWidth: 0.5000
Color: [0 0.4470 0.7410]

```

```

% PZPlot properties:
% https://it.mathworks.com/help/ident/ref/controllib.chart.pzplot-properties.html
% https://it.mathworks.com/matlabcentral/answers/2173352-change-marker-colour-of-pzplot
%
hPZP.Responses(1).MarkerSize = 12;
hPZP.Responses(1).LineWidth = 2;
hPZP.Responses(1).Color = 'r';
get(hPZP.AxesStyle(1))

```

```

Box: on
FontSize: 10
FontWeight: "normal"
FontAngle: "normal"
FontName: "Helvetica"
RulerColor: [0.3804 0.3804 0.3804]
GridVisible: on
GridColor: [0.1294 0.1294 0.1294]
GridLineWidth: 0.5000

```

```

% see 'doc pzplot' & 'doc pzoptions'
hPZP.AxesStyle(1).FontSize = 14;
hPZP.AxesStyle(1).GridColor = [0,0,0];

```

```
hPZP.XLabel(1)
```

```

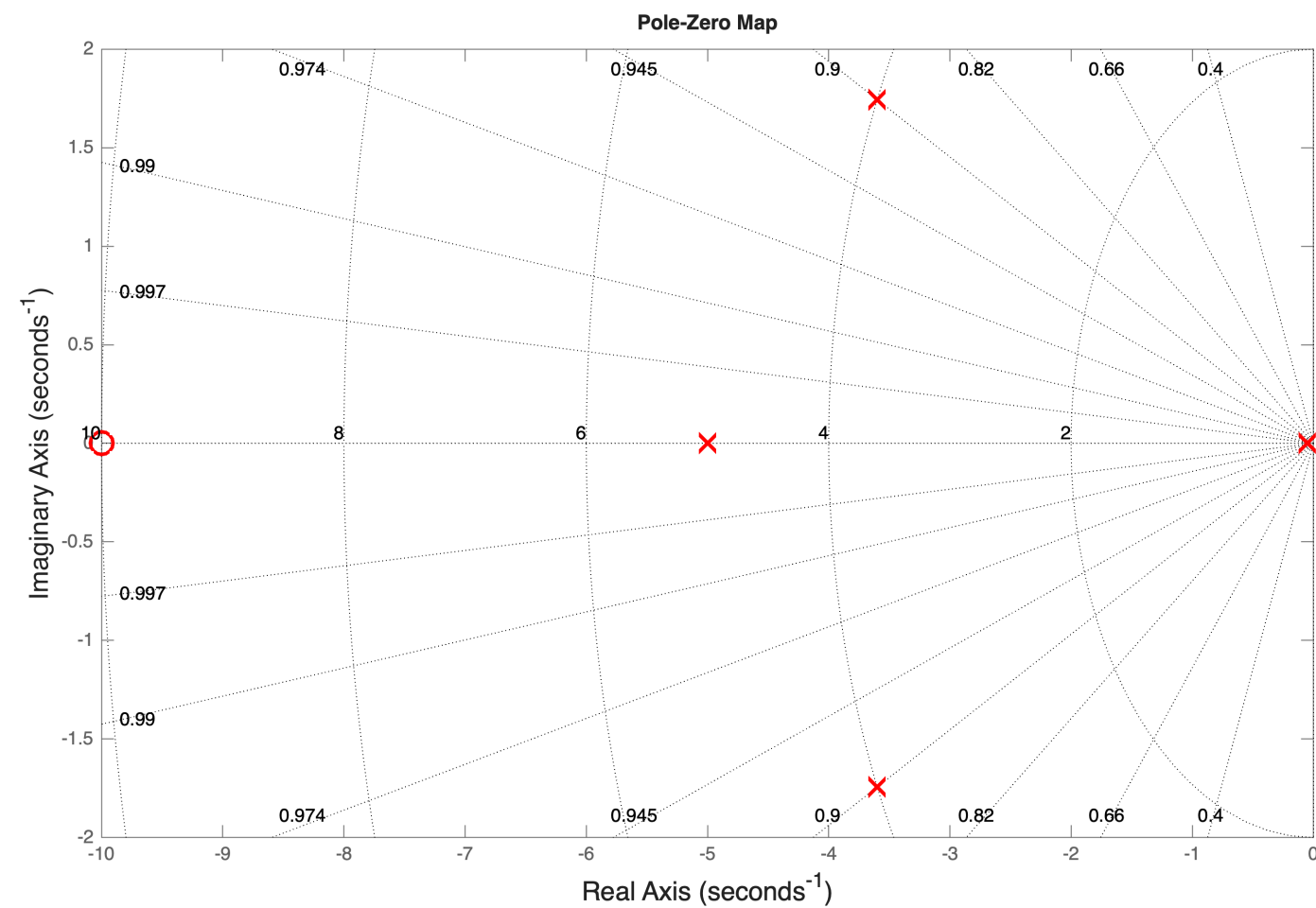
ans =
AxesLabel with properties:

    String: "Real Axis"
    FontSize: 11
    FontWeight: "normal"
    FontAngle: "normal"
    FontName: "Helvetica"
    Color: [0.1294 0.1294 0.1294]

```

Interpreter: "tex"
Rotation: 0
Visible: on

```
hPZP.XLabel(1).FontSize = 14;  
hPZP.YLabel(1).FontSize = 14;
```



The pole closest to the origin is the dominant one.

The Dominant Pole Approximation

We may approximate the system $G_1(s)$ using a very simple model: the static gain and the real dominant pole:

$$G_{1\text{ app}}(s) = 250 \cdot \frac{1}{(1 + 20s)}$$

```
tau1 = abs(1/p_G1s(1))
```

```
tau1 =  
20
```

```
G1sApprox = tf([G1_staticGAIN], [tau1 1], "Name", "G_approx")
```

```
G1sApprox =
    250
-----
 20 s + 1

Name: G_approx
Continuous-time transfer function.
Model Properties
```

Let's compare the step response of system $G_1(s)$ with the approximate model:

```
hf = figure('Units', 'normalized', 'Position', [0.1, 0.1, 0.9, 0.9]);
% let's create a blank figure

rpSP = stepplot(G1s, G1sApprox, [0, 180]);
rpSP
```

```
rpSP =
  StepPlot with properties:

    Responses: [2x1 controllib.chart.response.StepResponse]
  Characteristics: [1x1 controllib.chart.options.CharacteristicsManager]

    TimeUnit: "seconds"
    Normalize: off

    Visible: on
    IOGrouping: "none"
    InputVisible: on
    OutputVisible: on

  Show all properties
```

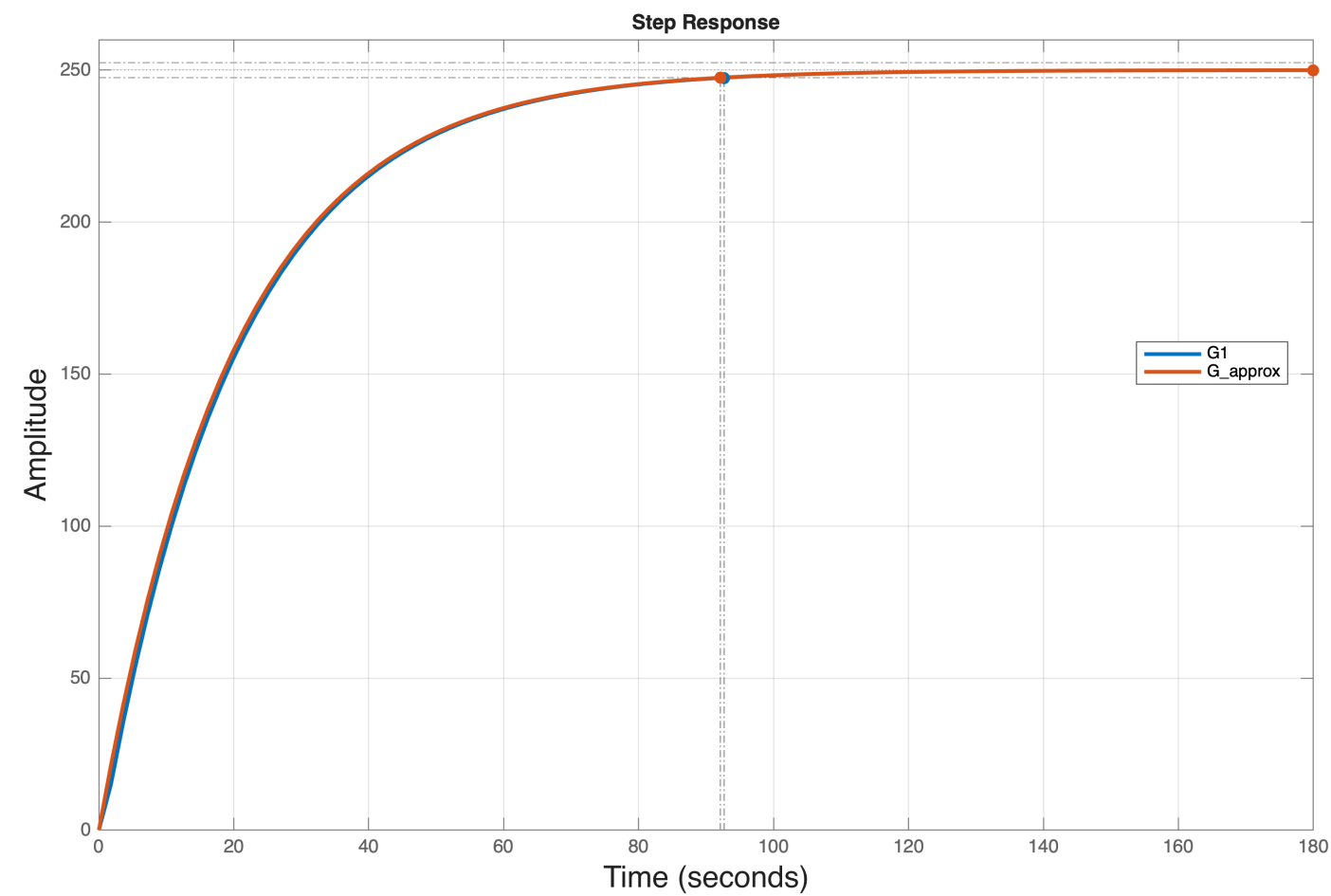
```
% refer to https://it.mathworks.com/help/control/ug/customizing-response-plots-from-the-command-line.html
rpSP.Characteristics(1).SettlingTime(1).Threshold = 0.01; % 1 %
rpSP.Characteristics(1).SettlingTime(1).Visible = "on";
rpSP.Characteristics(1).SteadyState(1).Visible = "on";
rpSP.LegendVisible = "on";
rpSP.LegendLocation = "best";

grid on;
rpSP.Responses
```

```
ans =
  2x1 StepResponse array:

    StepResponse    (G1)
    StepResponse    (G_approx)
```

```
rpSP.Responses(1).LineWidth = 2;
rpSP.Responses(2).LineWidth = 2;
ylim([0, 260]);
rpSP.AxesStyle(1).FontSize=16;
rpSP.XLabel(1).FontSize = 16;
rpSP.YLabel(1).FontSize = 16;
```



```
G1info = stepinfo(G1s, "SettlingTimeThreshold", 0.01)
```

```
G1info = struct with fields:
    RiseTime: 43.9474
    TransientTime: 92.6612
    SettlingTime: 92.6612
    SettlingMin: 225.4562
    SettlingMax: 249.8302
    Overshoot: 0
    Undershoot: 0
    Peak: 249.8302
    PeakTime: 146.4444
```

```
G1APPROXinfo = stepinfo(G1sApprox, "SettlingTimeThreshold", 0.01)
```

```
G1APPROXinfo = struct with fields:
    RiseTime: 43.9401
    TransientTime: 92.1034
    SettlingTime: 92.1034
    SettlingMin: 226.1252
    SettlingMax: 249.8348
    Overshoot: 0
    Undershoot: 0
    Peak: 249.8348
    PeakTime: 146.4444
```

Example 2 - A Real Dominant Pole and a Zero close to the Origin

Consider the LTI system described by the transfer function

$$G_2(s) = \frac{250 (1 + 50 s)}{(1 + 20 s) \left(1 + \frac{s}{5}\right) \left(1 + \frac{9}{20} s + \frac{s^2}{16}\right)}$$

```
clear
s = tf('s'); % the transfer function building helper

G2s = 250*(1+50*s)/((1+20*s)*(1+s/5)*(1+(9/20)*s+s^2/16)); % the tranfer function
set(G2s,"Name", "G2");

G2_staticGAIN = dcgain(G2s) % the static gain of the LTI system
```

G2_staticGAIN =
250

```
p_G2s = sort(pole(G2s), 'ascend') % the poles of the transfer function
```

p_G2s = 4×1 complex
-0.0500 + 0.0000i
-3.6000 - 1.7436i
-3.6000 + 1.7436i
-5.0000 + 0.0000i

```
z_G2s = zero(G2s) % the zeros of the transfer function
```

z_G2s =
-0.0200

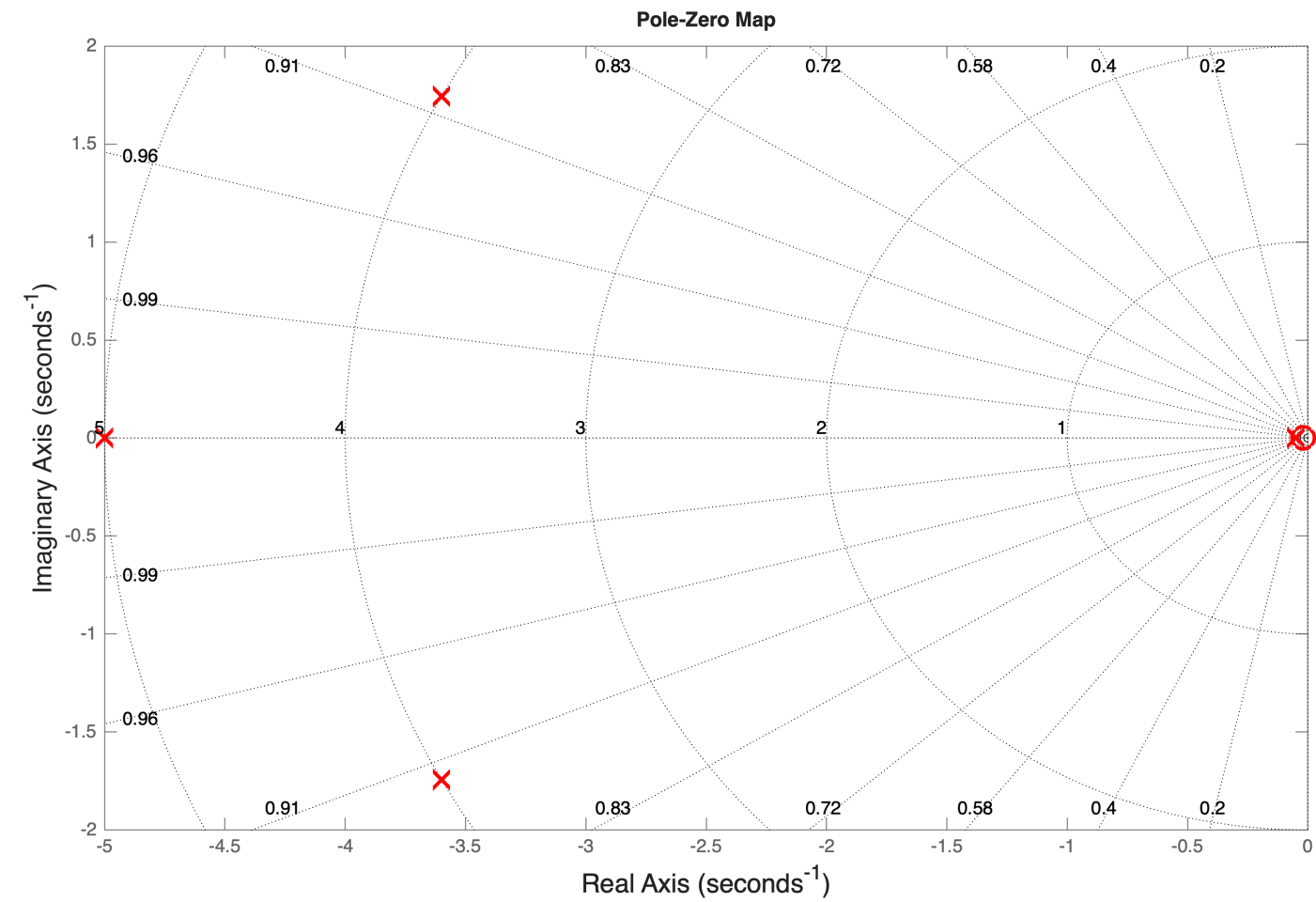
The poles and zeros configuration of the system is as follows

```
hf = figure('Units','normalized', 'Position', [0.1, 0.1, 0.9, 0.9]);
hPZP = pzplot(hf, G2s);
grid on
% get(hPZP)

% PZPlot properties:
% https://it.mathworks.com/help/ident/ref/ctrltolib.chart.pzplot-properties.html
% https://it.mathworks.com/matlabcentral/answers/2173352-change-marker-colour-of-pzplot
%
hPZP.Responses(1).MarkerSize = 12;
hPZP.Responses(1).LineWidth = 2;
hPZP.Responses(1).Color = 'r';
% get(hPZP.AxesStyle(1))

% see 'doc pzplot' & 'doc pzoptions'
hPZP.AxesStyle(1).FontSize = 14;
hPZP.AxesStyle(1).GridColor = [0,0,0];
```

```
% hPZP.XLabel(1)
hPZP.XLabel(1).FontSize = 14;
hPZP.YLabel(1).FontSize = 14;
```



The pole closest to the origin is the dominant one. The zero close to the origin is also crucial to obtain a good apprimating model for the system.

The Dominant Pole Approximation

We may approximate the system $G_2(s)$ using a very simple model: the static gain, the real dominant pole and the zero:

$$G_{2\text{ app}}(s) = 250 \cdot \frac{(1 + 50 s)}{(1 + 20 s)}$$

```
tau2 = abs(1/p_G2s(1))
```

```
tau2 =  
20
```

```
T2 = abs(1/z_G2s(1))
```

```
T2 =  
50
```

```
G2sApproxV1 = tf(G2_staticGAIN, [tau2 1], "Name", "G2_apprV1") % without the zero
```

G2sApproxV1 =

$$\frac{250}{20 s + 1}$$

Name: G2_apprV1

Continuous-time transfer function.

Model Properties

```
G2sAPPROX = G2_staticGAIN * tf([T2 +1], [tau2 +1]) % with the zero
```

G2sAPPROX =

$$\frac{12500 s + 250}{20 s + 1}$$

Continuous-time transfer function.

Model Properties

```
set(G2sAPPROX , "Name", "G2_apprV2")
```

Let's compare the step response of system $G_2(s)$ with the approximate models:

```
hf = figure('Units', 'normalized', 'Position', [0.1, 0.1, 0.9, 0.9]);  
% let's create a blank figure
```

```
rpSP2 =stepplot(G2s,G2sApproxV1, G2sAPPROX, [0, 180]);  
grid on;  
rpSP2
```

rpSP2 =
StepPlot with properties:

Responses: [3×1 controllib.chart.response.StepResponse]
Characteristics: [1×1 controllib.chart.options.CharacteristicsManager]

TimeUnit: "seconds"
Normalize: off

Visible: on
IOGrouping: "none"
InputVisible: on
OutputVisible: on

Show all properties

```
% refer to https://it.mathworks.com/help/control/ug/customizing-response-plots-from-the-command-line.html
```

```
rpSP2.Characteristics(1).SettlingTime(1).Threshold = 0.01; % 1 %  
rpSP2.Characteristics(1).SettlingTime(1).Visible = "on";  
rpSP2.Characteristics(1).SteadyState(1).Visible = "on";  
rpSP2.LegendVisible = "on";  
rpSP2.LegendLocation = "best";
```

```
grid on;  
rpSP2.Responses
```

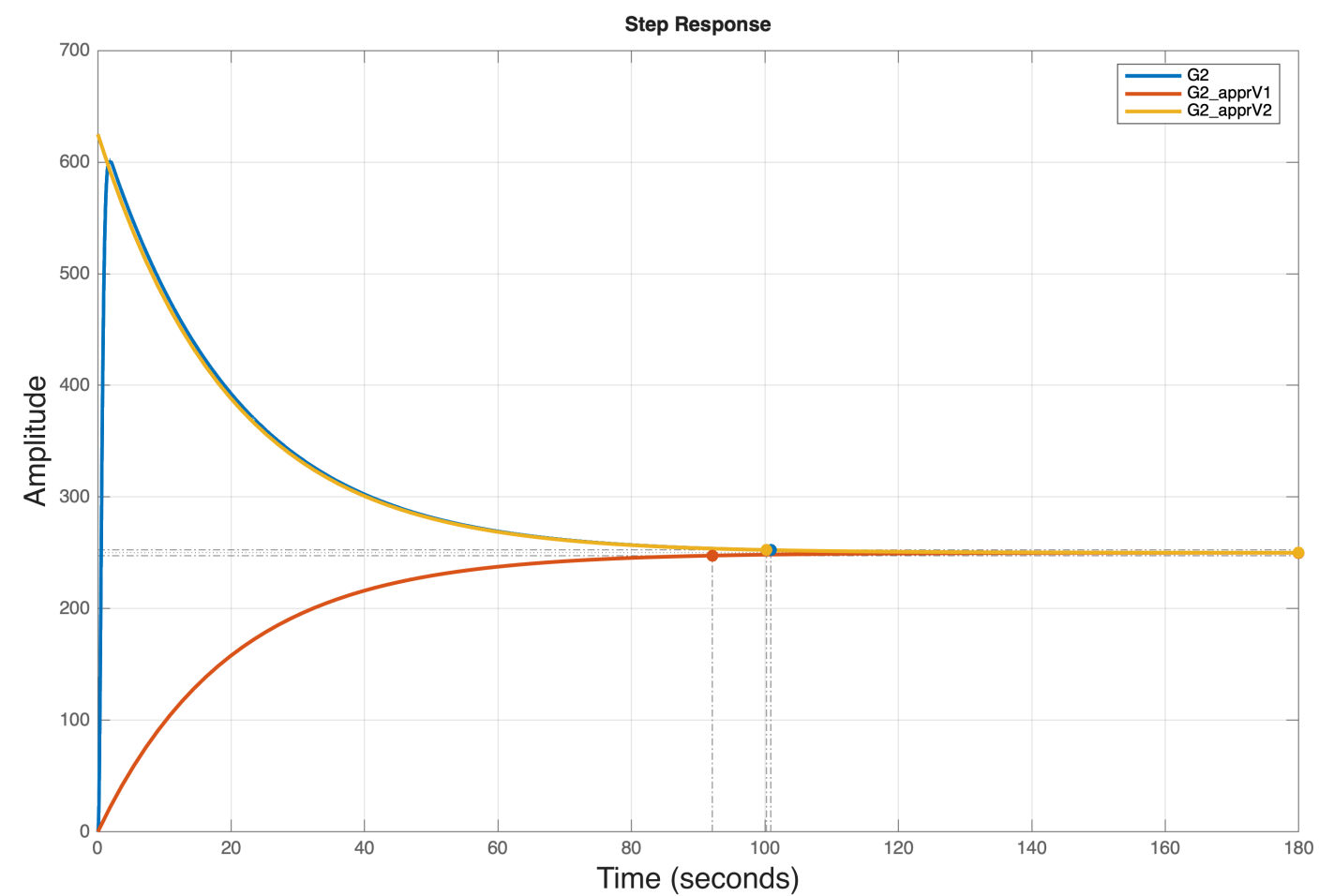
ans =

3×1 StepResponse array:

```
StepResponse (G2)  
StepResponse (G2_apprV1)  
StepResponse (G2_apprV2)
```

```
rpSP2.Responses(1).LineWidth = 2;  
rpSP2.Responses(2).LineWidth = 2;  
rpSP2.Responses(3).LineWidth = 2;
```

```
rpSP2.AxesStyle(1).FontSize=16;  
rpSP2.XLabel(1).FontSize = 16;  
rpSP2.YLabel(1).FontSize = 16;
```



```
G2info = stepinfo(G2s, "SettlingTimeThreshold", 0.01)
```

```
G2info = struct with fields:  
RiseTime: 0.3101  
TransientTime: 94.0374  
SettlingTime: 100.8657  
SettlingMin: 237.3440  
SettlingMax: 601.7330  
Overshoot: 140.6932  
Undershoot: 0  
Peak: 601.7330  
PeakTime: 1.7868
```

```
G2approx_V1_info = stepinfo(G2sApproxV1, "SettlingTimeThreshold", 0.01)
```

```
G2approx_V1_info = struct with fields:
    RiseTime: 43.9401
    TransientTime: 92.1034
    SettlingTime: 92.1034
    SettlingMin: 226.1252
    SettlingMax: 249.8348
    Overshoot: 0
    Undershoot: 0
    Peak: 249.8348
    PeakTime: 146.4444
```

```
G2approx_info = stepinfo(G2sAPPROX, "SettlingTimeThreshold", 0.01)
```

```
G2approx_info = struct with fields:
    RiseTime: 0
    TransientTime: 92.1034
    SettlingTime: 100.2160
    SettlingMin: 250.2478
    SettlingMax: 625
    Overshoot: 150
    Undershoot: 0
    Peak: 625
    PeakTime: 0
```

Example 3 - Complex Dominant Poles

Consider the LTI system described by the transfer function

$$G_3(s) = \frac{250 \left(1 + \frac{s}{40}\right)}{\left(1 + \frac{s}{20}\right) \left(1 + \frac{s}{15}\right) \left(1 + \frac{9}{20}s + \frac{s^2}{16}\right)}$$

```
clear
s = tf('s'); % the transfer function building helper

G3s = 250*(1+(s/40))/( (1+(s/20))*(1+s/15)*(1+(9/20)*s+s^2/16) ); % the transfer function

set(G3s, "Name", "G3");
G3_staticGAIN = dcgain(G3s) % the static gain of the LTI system
```

```
G3_staticGAIN =
250
```

```
p_G3s = sort(pole(G3s), 'ascend') % the poles of the transfer function
```

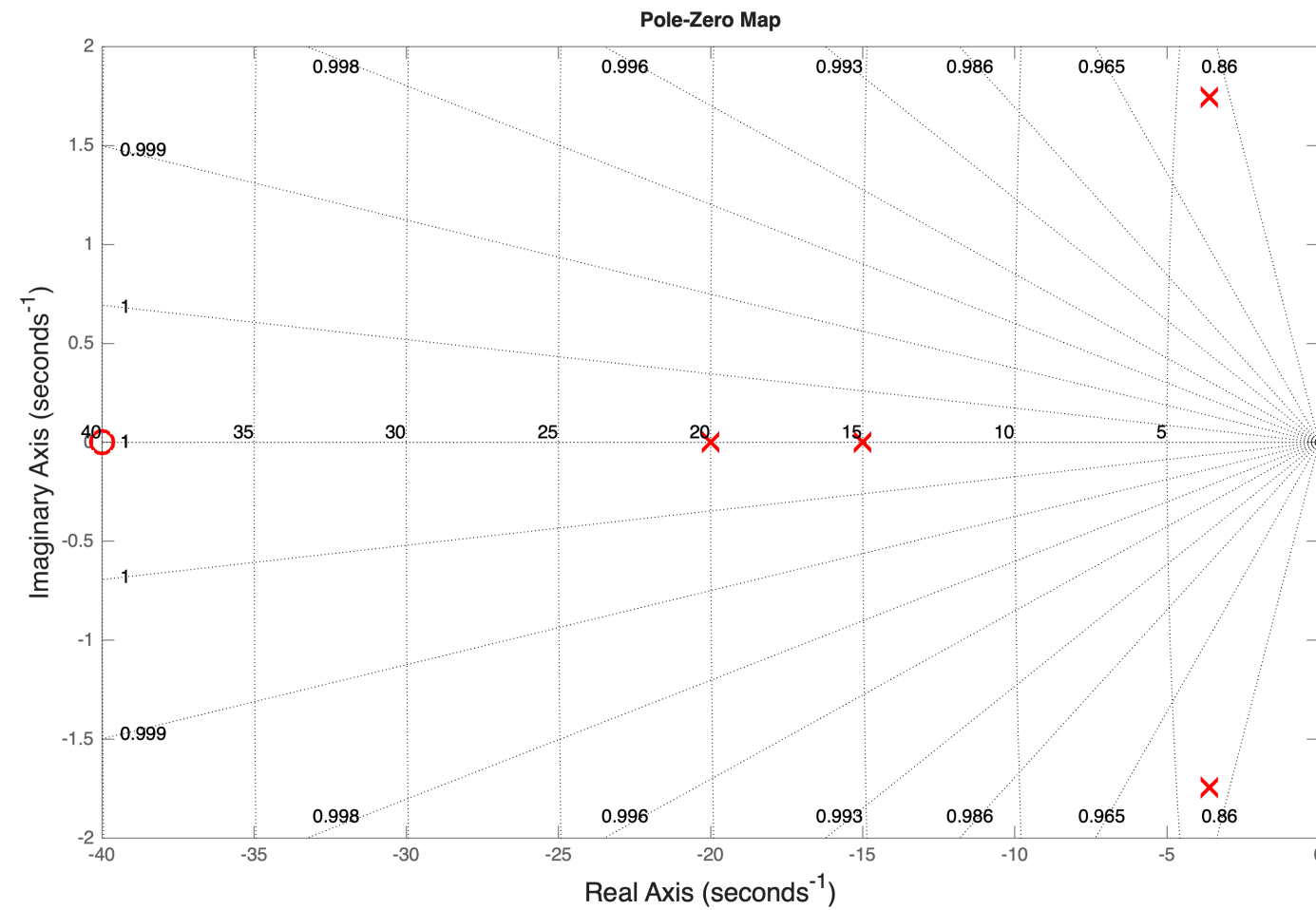
```
p_G3s = 4x1 complex
-3.6000 - 1.7436i
-3.6000 + 1.7436i
-15.0000 + 0.0000i
-20.0000 + 0.0000i
```

```
z_G3s = zero(G3s) % the zeros of the transfer function
```

```
z_G3s =  
-40
```

The poles and zeros configuration of the system is as follows

```
hf = figure('Units','normalized', 'Position', [0.1, 0.1, 0.9, 0.9]);  
hPZP3 = pzplot(hf, G3s);  
grid on  
% get(hPZP)  
  
% PZPlot properties:  
% https://it.mathworks.com/help/ident/ref/ctrltolib.chart.pzplot-properties.html  
% https://it.mathworks.com/matlabcentral/answers/2173352-change-marker-colour-of-pzplot  
%  
hPZP3.Responses(1).MarkerSize = 12;  
hPZP3.Responses(1).LineWidth = 2;  
hPZP3.Responses(1).Color = 'r';  
% get(hPZP.AxesStyle(1))  
  
% see 'doc pzplot' & 'doc pzoptions'  
hPZP3.AxesStyle(1).FontSize = 14;  
hPZP3.AxesStyle(1).GridColor = [0,0,0];  
  
% hPZP.XLabel(1)  
hPZP3.XLabel(1).FontSize = 14;  
hPZP3.YLabel(1).FontSize = 14;
```



The poles closest to the origin are the dominant ones.

The Dominant Poles Approximation

We may approximate the system $G_3(s)$ using a very simple model: the static gain and the dominant poles

$$G_{3\text{ app}}(s) = 250 \cdot \frac{1}{\left(1 + \frac{9}{20}s + \frac{s^2}{16}\right)}$$

```
G3sAPPROX = 250*(1)/( (1+(9/20)*s+s^2/16) ); % the approximating tranfer function
set(G3sAPPROX, "Name", "G3_approx");
```

Let's compare the step response of system $G_3(s)$ with the approximate model:

```
hf = figure('Units', 'normalized', 'Position', [0.1, 0.1, 0.9, 0.9]);
% let's create a blank figure

rpSP3 = stepplot(G3s, G3sAPPROX, [0, 180]);
rpSP3
```

```
rpSP3 =
```

StepPlot with properties:

Responses: [2×1 controllib.chart.response.StepResponse]
Characteristics: [1×1 controllib.chart.options.CharacteristicsManager]

TimeUnit: "seconds"
Normalize: off

Visible: on
IOGrouping: "none"
InputVisible: on
OutputVisible: on

Show all properties

```
% refer to https://it.mathworks.com/help/control/ug/customizing-response-plots-from-the-command-line.html
```

```
rpSP3.Characteristics(1).SettlingTime(1).Threshold = 0.01; % 1 %
```

```
rpSP3.Characteristics(1).SettlingTime(1).Visible = "on";
```

```
rpSP3.Characteristics(1).SteadyState(1).Visible = "on";
```

```
rpSP3.LegendVisible = "on";
```

```
rpSP3.LegendLocation = "best";
```

```
grid on;
```

```
rpSP3.Responses
```

```
ans =
```

```
2×1 StepResponse array:
```

```
StepResponse    (G3)
```

```
StepResponse    (G3_approx)
```

```
rpSP3.Responses(1).LineWidth = 2;
```

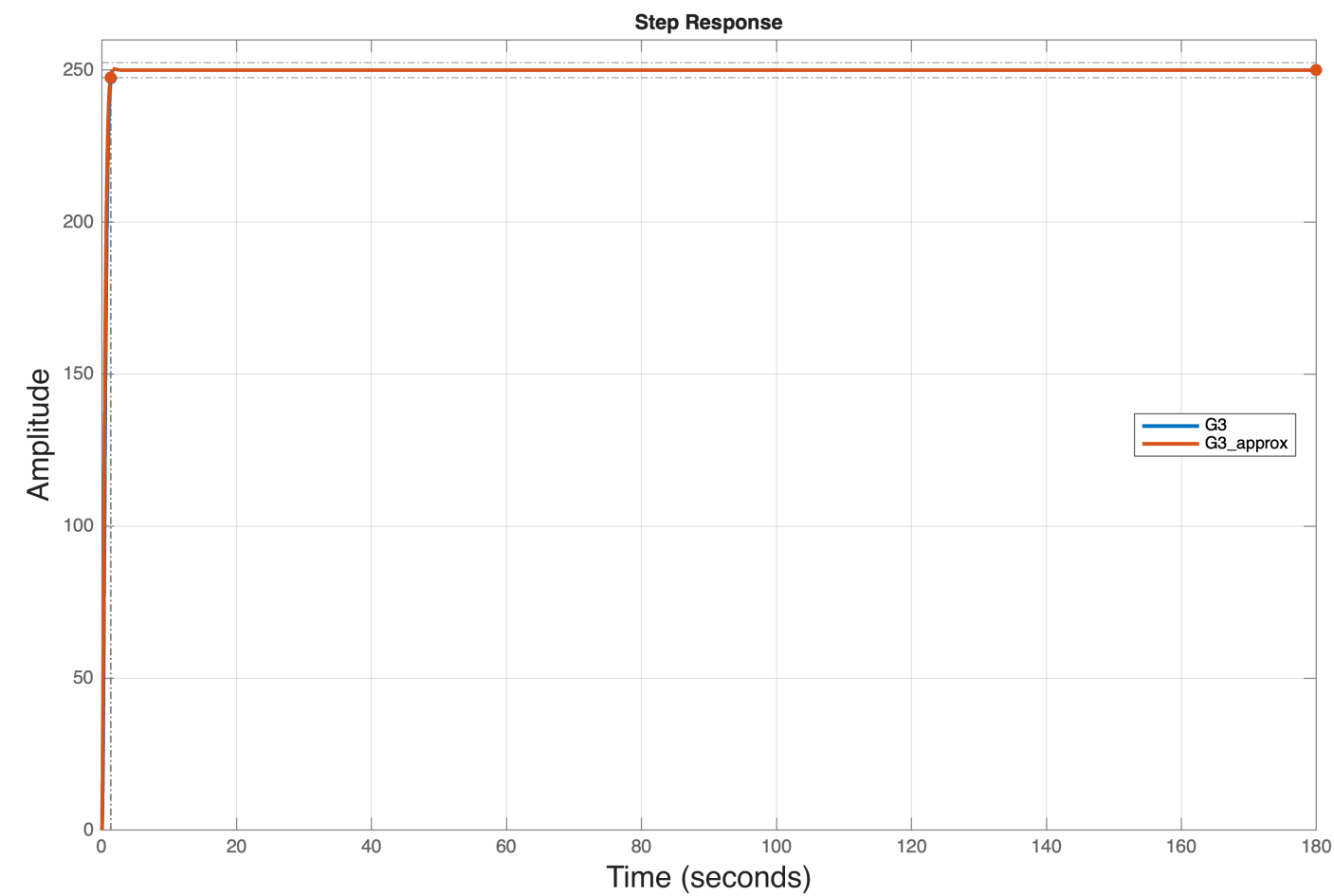
```
rpSP3.Responses(2).LineWidth = 2;
```

```
ylim([0, 260]);
```

```
rpSP3.AxesStyle(1).FontSize=16;
```

```
rpSP3.XLabel(1).FontSize = 16;
```

```
rpSP3.YLabel(1).FontSize = 16;
```



```
G3info = stepinfo(G3s, "SettlingTimeThreshold", 0.01)
```

```
G3info = struct with fields:
    RiseTime: 0.7496
    TransientTime: 1.3922
    SettlingTime: 1.3922
    SettlingMin: 225.4575
    SettlingMax: 250.3549
    Overshoot: 0.1420
    Undershoot: 0
    Peak: 250.3549
    PeakTime: 1.9188
```

```
G3approx_info = stepinfo(G3sAPPROX, "SettlingTimeThreshold", 0.01)
```

```
G3approx_info = struct with fields:
    RiseTime: 0.7209
    TransientTime: 1.2819
    SettlingTime: 1.2819
    SettlingMin: 225.6020
    SettlingMax: 250.3809
    Overshoot: 0.1524
    Undershoot: 0
    Peak: 250.3809
    PeakTime: 1.8037
```

Final Example - A Set of Dominant Poles and Zeros

Consider the LTI system described by the transfer function

$$G_4(s) = \frac{250 \left(1 + \frac{4}{5}s + \frac{s^2}{4}\right) \left(1 + \frac{s}{200}\right)}{\left(1 + \frac{s}{20}\right) \left(1 + \frac{s}{3}\right) \left(1 + \frac{9}{20}s + \frac{s^2}{16}\right)}$$

```
clear
s = tf('s'); % the transfer function building helper

G4s = 250*(1+0.8*s+(s^2/4))*(1+s/200)/((1+(s/20))*(1+s/3)*(1+(9/20)*s+s^2/16)); % the tranfer function
set(G4s, "Name", "G4");

G4_staticGAIN = dcgain(G4s) % the static gain of the LTI system
```

G4_staticGAIN =
250

```
p_G4s = sort(pole(G4s), 'ascend') % the poles of the transfer function
```

p_G4s = 4x1 complex
-3.0000 + 0.0000i
-3.6000 - 1.7436i
-3.6000 + 1.7436i
-20.0000 + 0.0000i

```
z_G4s = sort(zero(G4s), 'ascend') % the zeros of the transfer function
```

z_G4s = 3x1 complex
10² ×
-0.0160 - 0.0120i
-0.0160 + 0.0120i
-2.0000 + 0.0000i

The poles and zeros configuration of the system is as follows

```
figure;
hPZP4 = pzplot(G4s);
grid on

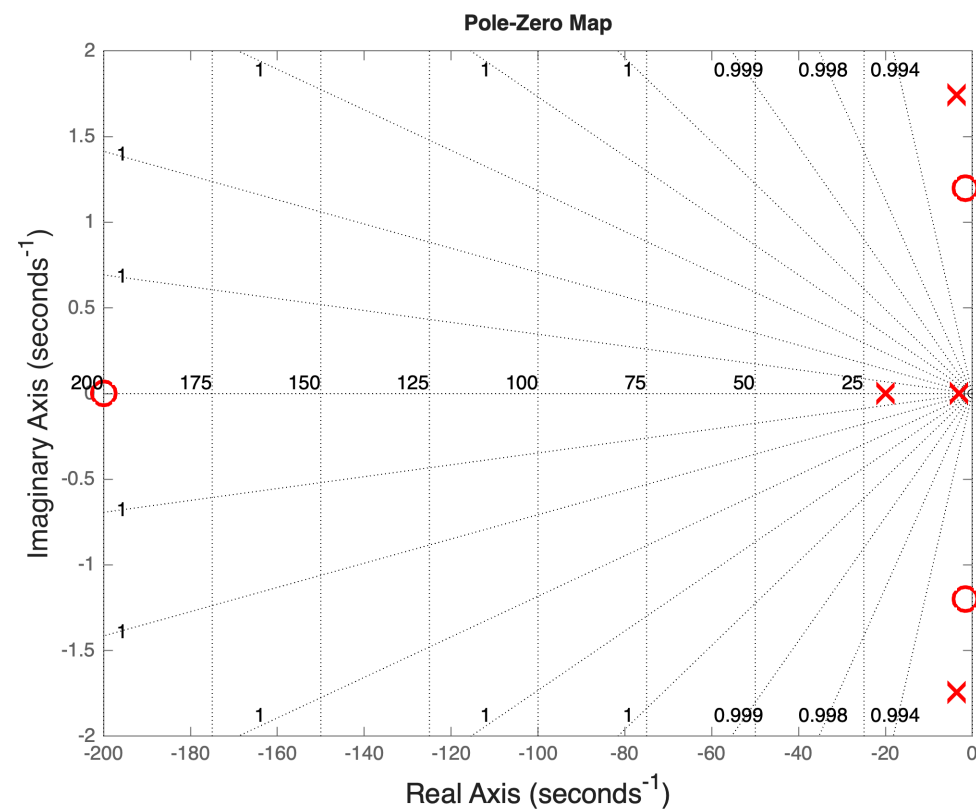
% get(hPZP)

% PZPlot properties:
% https://it.mathworks.com/help/ident/ref/controllib.chart.pzplot-properties.html
% https://it.mathworks.com/matlabcentral/answers/2173352-change-marker-colour-of-pzplot
%
hPZP4.Responses(1).MarkerSize = 12;
hPZP4.Responses(1).LineWidth = 2;
```

```
hPZP4.Responses(1).Color = 'r';
% get(hPZP.AxesStyle(1))

% see 'doc pzplot' & 'doc pzoptions'
hPZP4.AxesStyle(1).FontSize = 14;
hPZP4.AxesStyle(1).GridColor = [0,0,0];

% hPZP.XLabel(1)
hPZP4.XLabel(1).FontSize = 14;
hPZP4.YLabel(1).FontSize = 14;
```



The poles closest to the origin are the dominant ones. This time, there are two complex poles and one real pole close to the origin (with real parts very close to each other - we cannot neglect any of the poles in question). In addition, the pair of complex zeros must also be considered.

The Dominant Poles Approximation

We may approximate the system $G_4(s)$ using a very simple model: the static gain and the dominant poles

$$G_{4\text{ app}}(s) = 250 \cdot \frac{\left(1 + \frac{4}{5}s + \frac{s^2}{4}\right)}{\left(1 + \frac{s}{3}\right)\left(1 + \frac{9}{20}s + \frac{s^2}{16}\right)}$$

```
G4sAPPROX = 250*(1+0.8*s+(s^2/4))/( (1+s/3)*(1+(9/20)*s+s^2/16) ); % the approximating tranfer function
```

```
set(G4sAPPROX, "Name", "G4_approx");
```

Let's compare the step response of system $G_3(s)$ with the approximate model:

```
hf = figure('Units', 'normalized', 'Position', [0.1, 0.1, 0.9, 0.9]);  
% let's create a blank figure
```

```
rpSP4 = stepplot(G4s, G4sAPPROX, [0, 180]);  
grid on; zoom on;  
rpSP4
```

```
rpSP4 =  
  StepPlot with properties:  
  
    Responses: [2×1 controllib.chart.response.StepResponse]  
  Characteristics: [1×1 controllib.chart.options.CharacteristicsManager]  
  
    TimeUnit: "seconds"  
    Normalize: off  
  
    Visible: on  
    IOLGrouping: "none"  
    InputVisible: on  
    OutputVisible: on
```

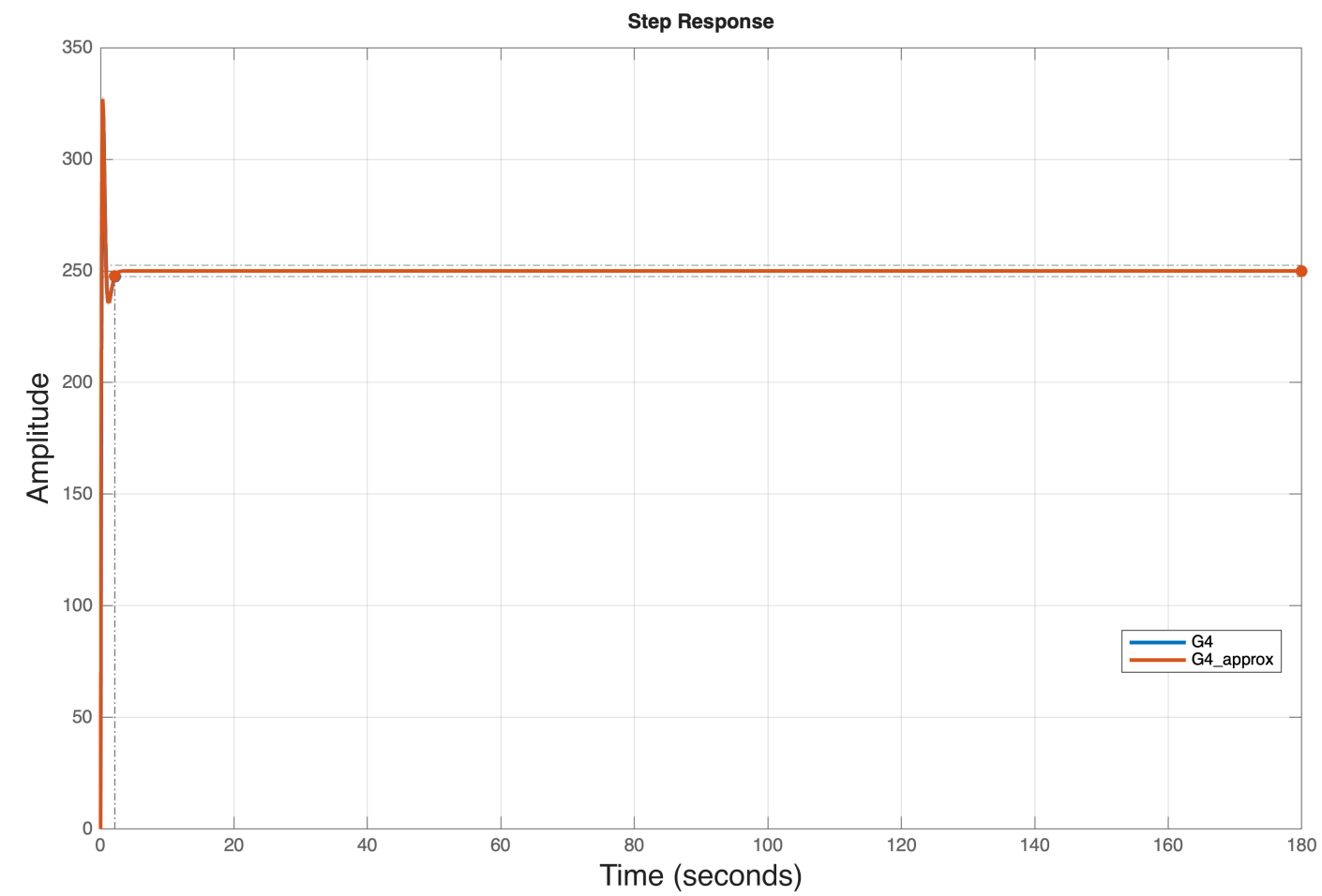
Show all properties

```
% refer to https://it.mathworks.com/help/control/ug/customizing-response-plots-from-the-command-line.html  
rpSP4.Characteristics(1).SettlingTime(1).Threshold = 0.01; % 1 %  
rpSP4.Characteristics(1).SettlingTime(1).Visible = "on";  
rpSP4.Characteristics(1).SteadyState(1).Visible = "on";  
rpSP4.LegendVisible = "on";  
rpSP4.LegendLocation = "best";
```

```
rpSP4.Responses
```

```
ans =  
  2×1 StepResponse array:  
  
  StepResponse      (G4)  
  StepResponse      (G4_approx)
```

```
rpSP4.Responses(1).LineWidth = 2;  
rpSP4.Responses(2).LineWidth = 2;  
%ylim([0, 260]);  
rpSP4.AxesStyle(1).FontSize=16;  
rpSP4.XLabel(1).FontSize = 16;  
rpSP4.YLabel(1).FontSize = 16;
```



```
G4info = stepinfo(G4s, "SettlingTimeThreshold", 0.01)
```

```
G4info = struct with fields:
    RiseTime: 0.1341
    TransientTime: 2.1588
    SettlingTime: 2.1588
    SettlingMin: 232.3190
    SettlingMax: 323.0583
    Overshoot: 29.2233
    Undershoot: 0
    Peak: 323.0583
    PeakTime: 0.3838
```

```
G4approx_info = stepinfo(G4sAPPROX, "SettlingTimeThreshold", 0.01)
```

```
G4approx_info = struct with fields:
    RiseTime: 0.1012
    TransientTime: 2.1106
    SettlingTime: 2.1106
    SettlingMin: 235.5384
    SettlingMax: 327.2357
    Overshoot: 30.8943
    Undershoot: 0
    Peak: 327.2357
    PeakTime: 0.3224
```

